

جستجو و کاوش آگاهانه

در این فصل، می‌بینیم که چگونه اطلاعاتی درباره فضای حالت می‌تواند از حرکت کورکورانه الگوریتمها در تاریکی جلوگیری کند.

ف $\Rightarrow$ E	Exploration	کاوش
	Inefficient	ناکارا
	Local search	جستجوی محلی
	Statistical Physics	فیزیک آماری
	Simulated annealing	شبه تاب کاری
	Evolutionary biology	زیست‌شناسی
		تکاملی
	Online search	جستجوی برخط
	Best-first search	جستجوی اول‌بهترین
		اول‌بهترین
	Evaluation function	تابع ارزیاب

فصل ۳ نشان داد که راهبردهای جستجوی ناآگاهانه می‌توانند راه‌حل مسائل را با تولید بسامان حالت‌های جدید و آزمودن آنها در مقابل هدف، بیابند. متأسفانه در اکثر موارد، این راهبردها به طور باورنکردنی ناکارا هستند. این فصل نشان می‌دهد که چگونه یک راهبرد جستجوی آگاهانه (که از دانش خاص مسأله استفاده می‌کند) می‌تواند به طور مؤثرتری راه‌حلها را بیابد. بخش ۴-۱ نسخه‌های آگاهانه الگوریتمهای فصل ۳ را توصیف می‌کند و بخش ۴-۲ توضیح می‌دهد که چگونه اطلاعات لازم خاص مسأله را می‌توان به دست آورد. بخشهای ۴-۳ و ۴-۴ الگوریتمهایی را دربرمی‌گیرند که به جای کاوش بسامان مسیرها از حالت اولیه، با ارزیابی و تغییر یک یا چند حالت فعلی، در فضای حالت جستجوی محلی انجام می‌دهند. این الگوریتمها در مورد مسائل مستقل از هزینه مسیر که در آنها حالت راه‌حل اهمیت دارد، مناسب هستند. فیزیک آماری (شبه تاب کاری) و زیست‌شناسی تکاملی (الگوریتمهای ژنتیک) الهام‌بخش خانواده الگوریتمهای جستجوی محلی است. نهایتاً بخش ۴-۵ جستجوی برخط را بررسی می‌کند که در آن، کارگزار با فضای حالتی کاملاً ناشناخته روبروست.

۱-۴ راهبردهای جستجوی آگاهانه (آروینی) (هیدرستیک) *harvesting*

این بخش نشان می‌دهد که چگونه یک راهبرد جستجوی آگاهانه با استفاده از دانش خاص مسأله، که فراتر از تعریف آن مسأله است، می‌تواند نسبت به راهبرد ناآگاهانه راه‌حل‌های مؤثرتری بیابد. رویکرد کلی مورد نظر، جستجوی اول‌بهترین نامیده می‌شود. جستجوی اول‌بهترین نمونه‌ای از الگوریتم عمومی TREE-SEARCH یا GRAPH-SEARCH است که در آن یک گره براساس یک تابع ارزیاب، $f(n)$ جهت گسترش انتخاب می‌شود. به طور معمول گرهی با کمترین مقدار ارزیابی، برای گسترش انتخاب می‌شود. زیرا مقیاس ارزیابی، فاصله تا هدف را اندازه می‌گیرد. می‌توان جستجوی اول‌بهترین را به کمک یک صف اولویت (ساختار داده‌ای که لبه را به ترتیب صعودی مقادیر f نگهداری می‌کند) در چارچوب کلی جستجو پیاده‌سازی کرد.

نام جستجوی اول‌بهترین، نام دقیقی نیست. اگر ما واقعاً می‌توانستیم اول بهترین گره را گسترش دهیم، دیگر این یک جستجو نبود، بلکه پیش‌روی مستقیم به سوی هدف بود. تنها کاری که ما می‌توان انجام داد این است که گرهی انتخاب شود که براساس تابع ارزیابی به نظر می‌رسد، بهترین باشد. اگر تابع ارزیاب کاملاً دقیق باشد، آنگاه این گره واقعاً بهترین گره است. در واقعیت، گاهی اوقات ممکن است این تابع ارزیاب خراب باشد و می‌تواند جستجو را به بیراهه بکشاند. با وجود این، از همان نام "جستجوی اول‌بهترین" استفاده می‌کنیم، زیرا "جستجوی اول

به نظر بهترین" خیلی عنوان جالبی نیست! یک خانواده کامل از الگوریتمهای *BEST-FIRST-SEARCH* با توابع ارزیاب مختلف وجود دارد.^۱ یک جزء کلیدی این الگوریتمها، تابع آروینی^۲ می‌باشد که به شکل $h(n)$ نشان داده می‌شود:

$h(n)$: برآورد کم هزینه‌ترین مسیر از گره n تا گره هدف

برای مثال، در مسأله رومانی می‌توان کم هزینه‌ترین مسیر از *Arad* به *Bucharest* را با مسافت یک خط مستقیم از *Arad* به *Bucharest* برآورد کرد. توابع آروین معمول‌ترین شکل انتقال اطلاعات اضافی مسأله به الگوریتم جستجو هستند. در بخش ۲-۴ آروینها را عمیق‌تر مورد مطالعه قرار می‌دهیم. فعلاً آنها را به عنوان توابع خاص مسأله، با این محدودیت در نظر می‌گیریم که: اگر n گره هدف باشد، آنگاه $h(n)=0$ باقیمانده این بخش، دو روش استفاده از اطلاعات آروینی را برای راهنمایی جستجو دربرمی‌گیرد.

جستجوی اول بهترین حریصانه

جستجوی اول‌بهترین حریصانه^۳ سعی می‌کند که نزدیک‌ترین گره به هدف را گسترش دهد، به این دلیل که احتمال زیادی دارد که سریع‌ترین راه‌حل باشد. بنابراین گرهها را با استفاده از تابع آروین: $f(n) = h(n)$ ارزیابی می‌کند.

اکنون ببینیم که این روش در مورد مسأله مسیریابی در رومانی، با استفاده از آروینی مسافت مستقیم که h_{SLD} می‌نامیم چگونه عمل می‌کند. اگر هدف *Bucharest* باشد، لازم است مسافتهای مستقیم تا *Bucharest* را که در شکل ۱-۴ نشان داده شده است بدانیم. برای مثال، $h_{SLD}(In(Arad))=366$ توجه داشته باشید که مقادیر h_{SLD} با مسافتهای واقعی جاده‌ها ارتباط دارد و در نتیجه، یک آروین سودمند است.

شکل ۲-۴ پیشرفت جستجوی اول‌بهترین حریصانه که برای یافتن مسیری از *Arad* به *Bucharest* از h_{SLD} استفاده می‌کند را نشان می‌دهد. از *Arad*، اولین گرهی که گسترش می‌یابد، *Sibiu* است، زیرا از *Zerind* و از *Timisoara* به *Bucharest* نزدیک‌تر است. گره بعدی برای گسترش، *Fagaras* است، زیرا که نزدیک‌ترین است. *Fagaras* هم به نوبه خودش، *Bucharest* را تولید می‌کند که هدف است. در مورد این مسأله خاص، جستجوی اول‌بهترین

E	ف
Evaluation	ارزیابی
Expansion	گسترش
Distance to the goal	فاصله تا هدف
Priority queue	صف اولویت
Fringe	لبه
Ascending order	ترتیب صعودی
Framework	چارچوب
Evaluation function	تابع ارزیاب
Key component	جزء کلیدی
Heuristic function	تابع آروین
Estimated cost	هزینه برآورد
Problem-specific	خاص مسأله
Greedy best-first search	جستجوی اول‌بهترین حریصانه

^۱ تمرین ۳-۴ از شما می‌خواهد که نشان دهید این خانواده، چندین الگوریتم ناآگاهانه آشنا را نیز شامل می‌شود.

^۲ یک تابع آروین $h(n)$ یک گره را به عنوان ورودی می‌گیرد، ولی تنها به حالت در آن گره بستگی دارد.

^۳ در ویرایش اول این کتاب، روش مذکور را جستجوی حریصانه می‌نامیدیم. نویسندگان دیگر آن را جستجوی اول‌بهترین نامیده‌اند. دلیل ما برای استفاده از اصطلاح دومی به پیروی از کتاب *pearl (1984)* می‌باشد.

ف $\rightarrow$ E
Useful
سودمند
To expand
گسترش یافتن
In turn
به نوبه خود
Particular
problem
مسأله خاص
Cost of search
هزینه جستجو
Greedy
حریصانه
Close
نزدیک
To minimize
کمینه کردن
Solution
راه حل
False starts
شروعهای نادرست
First
ابتدا
Dead end
بن بست
Unnecessary
غیر ضروری
Repeated
states
گره های تکراری
To oscillate
نوسان کردن
Defects
نقایص
Depth-first
search
جستجوی اول عمق

حریصانه، با استفاده از $h(n)$ راه حل را بدون گسترش حتی یک گره خارج از مسیر راه حل، پیدا می کند. در نتیجه، هزینه جستجو حداقل است. در عین حال، بهینه نیست، مسیری که از طریق Sibiu و Fagaras به Bucharest می رسد ۳۲ کیلومتر طولانی تر از مسیری است که از Rimnicu Vilcea و Pitesti می گذرد. اکنون معلوم می شود که به چه علت این الگوریتم، حریصانه نامیده می شود (زیرا در هر گام سعی می کند تا حد امکان به هدف نزدیک شود).

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Dobreta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

شکل ۴-۱ مقادیر $h(n)$ مسافتهای مستقیم به Bucharest

کمینه کردن $h(n)$ ممکن است باعث شروعهای نادرست شود. مسأله رسیدن از Iasi به Fagaras را در نظر بگیرید. آروین پیشنهاد می کند که ابتدا Neamt گسترش پیدا کند، زیرا نزدیک ترین به Fagaras است اما این مسیر بن بست است. راه حل این است که ابتدا به Vaslui برویم (در حقیقت، این گام براساس آروین از هدف دورتر است) و سپس، مسیر را از Urziceni، Bucharest و Fagaras ادامه دهیم. در نتیجه، در این حالت آروین باعث گسترش گره های غیر ضروری می شود. علاوه بر این، اگر گره های تکراری را شناسایی نکنیم، راه حل پیدا نمی شود (جستجو مدام در حال نوسان بین Neamt و Iasi خواهد بود).

جستجوی اول بهترین حریصانه شبیه جستجوی اول عمق است، از این جهت که ترجیح می دهد یک مسیر را در تمام طول راه تا هدف دنبال کند. ولی اگر به بن بست برسد، به عقب برمی گردد. این روش از نقایصی رنج می برد که در جستجوی اول عمق نیز وجود داشت (بهینه نیست و غیر کامل است زیرا ممکن است با آغاز یک مسیر بی انتها، هرگز هیچ امکان دیگری را امتحان نکند). پیچیدگی زمانی و فضایی در بدترین حالت $O(b^m)$ است که m حداکثر عمق فضای جستجو است. در عین حال با اتخاذ یک تابع آروین خوب، پیچیدگی می تواند به مقدار قابل توجهی کاهش یابد. مقدار کاهش به مسأله و به کیفیت آروین بستگی دارد.

جستجوی A^* : کمینه کردن کل هزینه برآورد راه حل

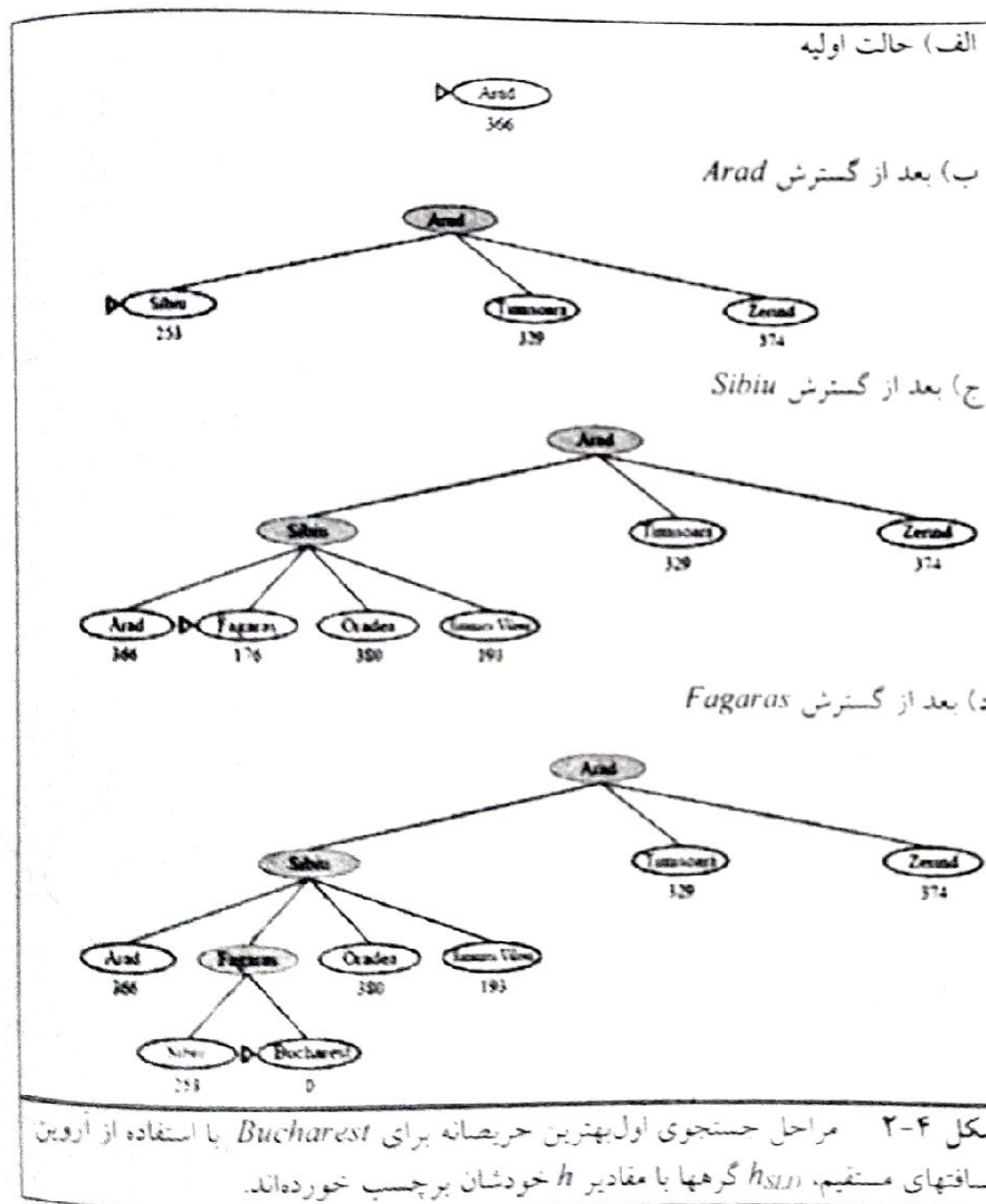
شناخته شده ترین شکل جستجوی اول بهترین، جستجوی A^* نامیده می شود (که جستجوی A ستاره خوانده می شود). این روش با ترکیب $g(n)$ ، هزینه رسیدن به آن گره و $h(n)$ هزینه رسیدن از آن گره به هدف، گره ها را ارزیابی می کند:

$$f(n) = g(n) + h(n)$$

از آنجا که $g(n)$ هزینه مسیر از گره شروع به گره n و $h(n)$ هزینه تخمینی ارزانترین مسیر از n به هدف است، داریم:

هزینه برآورد ارزانترین راه حل از طریق $f(n) = n$

بنابراین اگر به دنبال ارزانترین راه حل هستیم، اولین کار معقول امتحان گرهی است که کمترین مقدار $g(n) + h(n)$ را داراست. مشخص می شود که این راهبرد کاملاً معقولانه است. اگر تابع آروین $h(n)$ شرایط خاصی داشته باشد، جستجوی A^* هم کامل و هم بهینه است.



$E \Rightarrow$	ف
To minimize	کمینه کردن
Total estimated solution cost	کل هزینه برآورد
Known	راه حل شناخته شده
To evaluate	ارزیابی کردن
Cheapest solution	ارزانترین راه حل
Reasonable	معقول
Approach	راهبرد
Certain conditions	شرایط خاص
Complete	کامل
Optimal	بهینه
Greedy best-first search	جستجوی اول بهترین
Bucharest	خارست
Straight-line distance heuristic	آروین مسافتی مستقیم
Analysis	تحلیل

اگر A^* همراه با TREE-SEARCH استفاده شود، آنگاه تحلیل بهینگی آن بسیار ساده و سرراست می شود. در این حالت، A^* بهینه است اگر $h(n)$ یک آروین قابل قبول باشد (یعنی به شرطی که $h(n)$ هرگز هزینه رسیدن به هدف را بیش از اندازه واقعی برآورد نکند). آروینهای قابل قبول

$E \Rightarrow$	ف
straightforward	سراست
Admissible heuristic	آروین قابل قبول
To overestimate	بیش از اندازه واقعی
Obvious	برآورد کردن
Shortest path	واضح
Progress	کوتاه‌ترین مسیر
Overestimate	پیشرفت
Point	برآورد بیش از اندازه
General	نقطه
Suboptimal	کلی
Repeated state	نیمه‌بهینه
To guarantee	حالت تکراری
To occupy	تضمین کردن
Uniform-cost search	اشغال کردن
Consistency	جستجوی هزینه
Monotonicity	یکنواخت
Successor	سازگاری
	یکنواختی
	پسین

ذاتاً بهینه هستند، زیرا آنها فکر می‌کنند که هزینه راه‌حل مسأله کمتر از مقدار واقعی آن است. از آنجا که $g(n)$ هزینه رسیدن به n را به طور دقیق نشان می‌دهد، می‌توان بلافاصله نتیجه گرفت که $f(n)$ هرگز هزینه واقعی راه‌حلی که از n می‌گذرد را اضافه برآورد نمی‌کند.

یک مثال واضح از یک آروین قابل قبول، فاصله مستقیم h_{SLD} است که برای رسیدن به *Bucharest* استفاده کردیم. فاصله مستقیم، قابل قبول است زیرا کوتاه‌ترین مسیر بین هر دو نقطه، یک خط مستقیم است. بنابراین، آروین خط مستقیم نمی‌تواند یک برآورد بیش از اندازه باشد. شکل ۳-۴ پیشرفت یک درخت جستجوی A^* را برای رسیدن به *Bucharest* نشان می‌دهد. مقادیر g از هزینه گامهای شکل ۳-۲ محاسبه شده‌اند و مقادیر h_{SLD} در شکل ۳-۱ آمده‌اند. دقت کنید که *Bucharest* اولین بار در گام (ه) در لبه ظاهر می‌شود، اما برای گسترش انتخاب نمی‌شود، زیرا هزینه آن (یعنی ۴۵۰) از هزینه *Pitesti* (یعنی ۴۱۷) بیشتر است. از این مثال می‌توان یک نتیجه کلی گرفت که A^* می‌کند بهینه است اگر $h(n)$ قابل قبول باشد. فرض کنید که گره نیمه‌بهینه G_2 بر روی لبه ظاهر می‌شود و هزینه راه‌حل بهینه، C^* باشد. پس، از آنجا که G_2 نیمه‌بهینه است و $h(G_2) = 0$ (این جمله برای هر گره هدف، صحیح است)، می‌دانیم که

$$f(G_2) = g(G_2) + h(G_2) = g(G_2) > C^*$$

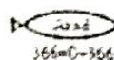
اکنون گره n که روی لبه است و در یک مسیر راه‌حل بهینه قرار دارد را در نظر بگیرید. (مثلاً *Pitesti* در مثال پاراگراف قبلی. اگر راه‌حلی وجود داشته باشد، همیشه چنین گرهی وجود دارد.) اگر $h(n)$ هزینه کامل کردن راه‌حل را بیش از اندازه برآورد نکند، در آن صورت می‌دانیم که

$$f(n) = g(n) + h(n) \leq C^*$$

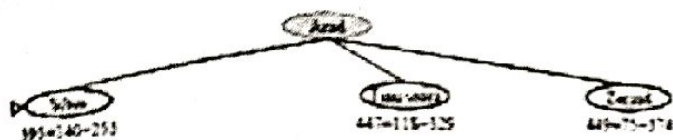
اکنون نشان داده‌ایم که $f(G_2) < C^* \leq f(n)$ بنابراین، G_2 گسترش نخواهد یافت و A^* باید یک راه‌حل بهینه را برگرداند.

اگر به جای *TREE-SEARCH* از *GRAPH-SEARCH* که در الگوریتم شکل ۳-۱۹ نشان داده شده بود، استفاده کنیم، آنگاه این قضیه دیگر صدق نمی‌کند و ممکن است راه‌حلهای نیمه‌بهینه برگردانده شوند. زیرا امکان دارد *GRAPH-SEARCH* میسر بهینه‌ای را که از یک حالت تکراری (گره‌ای که بار اول نیست که تولید می‌شود) می‌گذرد را کنار می‌گذارد. (به تمرین ۴-۴ رجوع کنید.) دو راه برای حل این مشکل وجود دارد. اولین راه‌حل این است که *GRAPH-SEARCH* را به نحوی توسعه بدهیم که از بین مسیرهایی که به یک گره می‌رسند آن مسیری که پرهزینه‌تر است را کنار بگذارد. (به مبحث بخش ۳-۵ مراجعه کنید.) این روش علی‌رغم اشغال فضای زیاد، بهینگی را تضمین می‌کند.

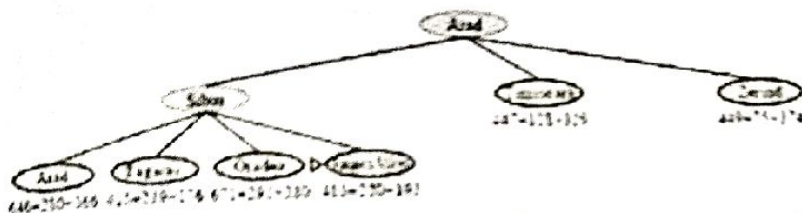
الف) حالت اولیه



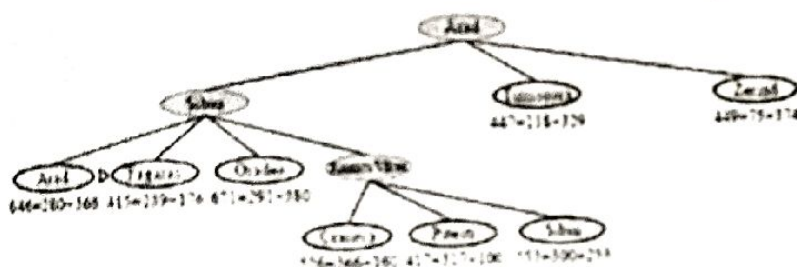
ب) بعد از گسترش Arad



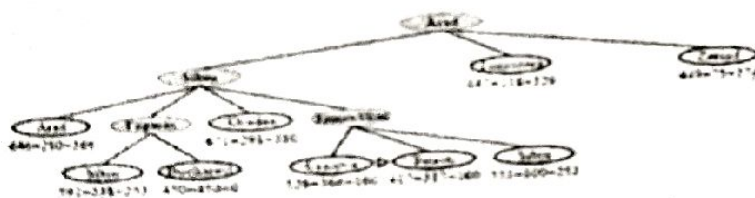
ج) بعد از گسترش Sibiu



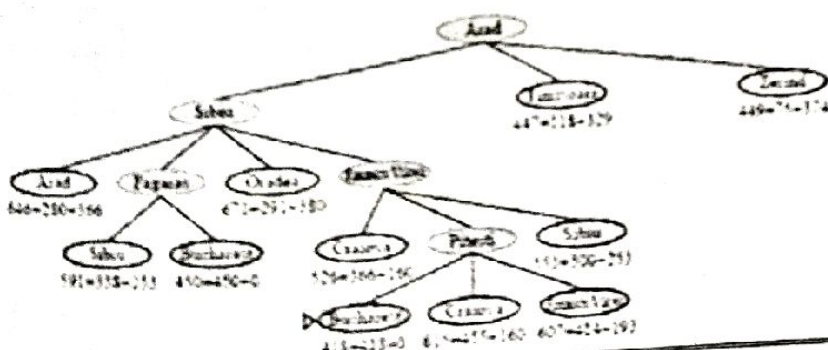
د) بعد از گسترش Rimnicu Vilcea



ه) بعد از گسترش Fagaras



و) بعد از گسترش Pitesti



ف $\Rightarrow$ E

General triangle inequality

قانون کلی نامساوی

مثلث

Side

ضلع

Stages

مراحل

Search

جستجو

Bucharest

بخارست

Node

گره

Label

برچسب

Value

مقدار

Straight-line distance

فاصله خط مستقیم

Figure

شکل

Long

بلند

Sum

مجموع

Closest goal

نزدیکترین هدف

Consistent heuristic

آروین سازگار

Admissible

قابل قبول

Consistency

سازگاری

Consequence

نتیجه

شکل ۳-۴ مراحل جستجوی A^* برای Bucharest. گرهها دارای برچسب $f=g+h$ هستند. مقادیر h فواصل خط مستقیم Bucharest هستند که از شکل ۱-۴ گرفته شده‌اند.

راهنما دوم. اطمینان از این موضوع است که مسیر بهینه‌ای که از هر حالت تکراری می‌گذرد، همیشه اولین مسیری است که دنبال می‌شود. همانند جستجوی هزینه یکنواخت. این ویژگی در

مسئله امتحان: روش A^* در گراف سرج در چه صورت بهینه است؟

۱۱۹

فصل ۴ - جستجو و کاوش آگاهانه

Consistency	ف E
Requirement	سازگاری
Directly	شرط
Definition	مستقیماً
Consistent	تعریف
Sequence of nodes	سازگار
Hence	در نتیجه
Nondecreasing	غیر نزولی
Contour	تراز
State space	فضای حالت
Topographic map	نقشه توپوگرافی
Label	برچسب
Concentric	نوارهای هم مرکز
Incremental	افزایشی
Finite	متناهی
Narrow	باریک
Optimal solution	راه حل بهینه
Equal	مساوی

صورتی به وجود می آید که یک شرط اضافی که شرط سازگاری (یا یکنواختی) نامیده می شود. به $h(n)$ نحلیل شود. آروین $h(n)$ سازگار است، اگر برای هر گره n و هر پسین آن گره، n' که با اقدام a تولید شده است، هزینه برآورد شده رسیدن به هدف از n بزرگتر از هزینه گام رسیدن به n' به علاوه هزینه برآورد شده رسیدن به هدف از n' نباشد:

$$h(n) \leq c(n, a, n') + h(n')$$

این رابطه، شکلی از قانون کلی نامساوی مثلث است که تصریح می کند هیچ ضلعی از یک مثلث نمی تواند بلندتر از مجموع دو ضلع دیگر باشد. در اینجا مثلث از n و n' و نزدیک ترین هدف به n تشکیل می شود. به سادگی می توان نشان داد که هر آروین سازگار، قابل قبول نیز هست (تمرین ۴-۷). مهم ترین نتیجه سازگاری به شرح ذیل است: A^* که از GRAPH-SEARCH استفاده می کند، بهینه است، در صورتی که $h(n)$ سازگار باشد.

اگر چه سازگاری نسبت به قابل قبول بودن شرط سخت گیرانه تری است، لیکن ساخت آروینهای قابل قبول ناسازگار، خیلی کار می برد. تمامی آروینهای قابل قبولی که در این فصل بحث می کنیم، سازگار نیز هستند. برای مثال، h_{SLD} را در نظر بگیرید. می دانیم که قانون کلی نامساوی مثلث در صورتی صحیح عمل می کند که هر ضلع را بر اساس فاصله مستقیم اندازه گیری کنیم و نیز می دانیم که فاصله مستقیم بین n و n' از $c(n, a, n')$ بزرگتر نیست. بنابراین، h_{SLD} یک آروین سازگار است.

نتیجه مهم دیگر سازگاری به شرح ذیل است: اگر $h(n)$ سازگار باشد، آنگاه مقدار $f(n)$ در طول هر مسیری، غیر نزولی است. این قضیه مستقیماً از تعریف سازگاری نتیجه می شود. فرض کنید n' یک پسین n است. آنگاه برای بعضی مقادیر a ، $g(n') = g(n) + c(n, a, n')$ و داریم:

$$f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n)$$

می توان این گونه نتیجه گرفت که رشته ای از گره ها که توسط A^* با استفاده از GRAPH-SEARCH گسترش یافته اند از نظر $f(n)$ غیر نزولی می باشند. در نتیجه، اولین گره هدفی که برای گسترش انتخاب می شود باید یک راه حل بهینه باشد، زیرا تمامی گره های بعدی حداقل به همان اندازه هزینه خواهند داشت.

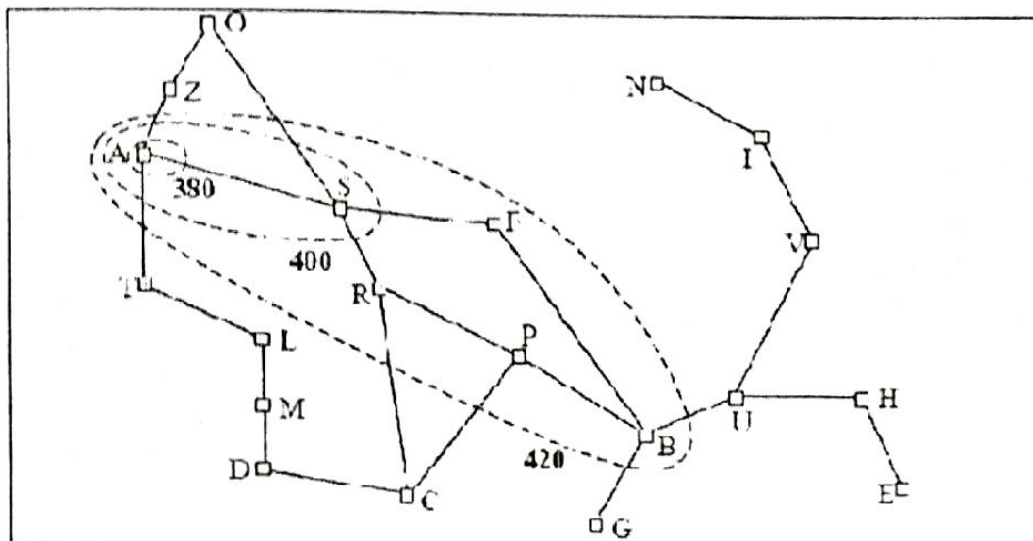
واقعیت غیر نزولی بودن هزینه های f در تمام طول مسیر، به این معنی نیز هست که همانند ترازهای یک نقشه توپوگرافی، می توان ترازهایی در فضای حالت رسم کرد. شکل ۴-۴ یک نمونه را نشان می دهد. درون تراز با برچسب ۴۰۰، تمامی گره ها دارای $f(n)$ کوچکتر مساوی ۴۰۰ هستند و به همین ترتیب، سپس به این علت که A^* گره لبه ای را که دارای کمترین هزینه است توسعه می دهد، می توان دریافت که جستجوی A^* از گره آغازین حرکت می کند و گره هایی با هزینه افزایشی را در نوارهای هم مرکز اضافه می کند.

با جستجوی هزینه یکنواخت (جستجوی A^* با استفاده از $h(n) = 0$) نوارهای پیرامون گره آغازین به شکل دایره خواهند بود. با آروینهای دقیق تر، نوارها به سمت گره هدف کشیده می شوند و به صورت باریک تری پیرامون مسیر بهینه تمرکز می یابند. اگر C^* هزینه راه حل بهینه باشد، می توان چنین اظهار کرد:

A^* تمامی گره هایی که دارای $f(n) < C^*$ هستند را توسعه می دهد.

مسئله امتحان: در گراف داده شده کدام نودها حتماً با روش A^* باز می شوند؟

A^* سپس ممکن است که بعضی از گره‌هایی را که دقیقاً بر روی تراز هدف قرار دارند (که $f(n) = C^*$) را قبل از انتخاب گره هدف، توسعه دهد. واضح است که اولین راه‌حل پیدا شده، باید یک راه‌حل بهینه باشد. زیرا گره‌های هدف در تمامی ترازهای بعدی دارای هزینه کمتری خواهند بود و در نتیجه هزینه g بالاتری نیز دارند (زیرا تمامی گره‌های هدف دارای $h(n) = 0$ هستند). همچنین، آشکار است که جستجوی A^* کامل است. همان‌طور که نوارهای با g در حال افزایش را اضافه می‌کنیم، باید بالاخره به نوار برسییم که در آنجا کم‌ترین هزینه مسیر تا یک حالت هدف است.^۴



شکل ۴-۴ نقشه رومانی که ترازهای $f=380$ ، $f=400$ و $f=420$ را نشان می‌دهد و حالت اولیه است. گره‌هایی که داخل یک تراز خاص قرار دارند، هزینه f کوچک‌تر یا مساوی مقدار تراز دارند.

دقت کنید که A^* هیچ گرهی را که دارای $f(n) > C^*$ باشد، توسعه نمی‌دهد (برای مثال در شکل ۴-۳، Timisoara توسعه پیدا نمی‌کند، اگر چه فرزند ریشه است). می‌گوییم که زیردرخت زیر Timisoara هرس می‌شود. از آنجا که h_{SLD} قابل قبول می‌باشد، الگوریتم می‌تواند از این زیردرخت صرف‌نظر کند. در حالی که هنوز بهینگی تضمین شده است. مفهوم هرس کردن (حذف احتمالات از فرضیه بدون نیاز به امتحان آنها) در بسیاری از زمینه‌های هوش مصنوعی دارای اهمیت است.

نکته آخر نیز این که در بین الگوریتم‌های بهینه‌ای از این نوع (الگوریتم‌هایی که مسیرهای جستجو را از ریشه توسعه می‌دهند) A^* برای هر تابع آروینی بهینه مؤثر است. یعنی هیچ الگوریتم بهینه دیگری تضمین نمی‌کند که تعداد گره‌هایی که توسعه می‌دهد از A^* کمتر باشد (به جز احتمالاً از طریق رفع تساوی بین گره‌هایی که $f(n) = C^*$ دارند). علت این امر آن است که هر الگوریتمی که تمامی گره‌ها را با $f(n) < C^*$ توسعه ندهد، خطر از دست دادن راه‌حل

^۴ کامل بودن نیاز به این دارد که تعداد گره‌های با هزینه کوچک‌تر مساوی C^* متناهی باشد. این شرط در صورتی صحیح است که تمامی هزینه‌های گام از مقدار متناهی E بیشتر و نیز b متناهی باشد.

ف	E
فرزند	Child
ریشه	Root
زیردرخت	Subtree
هرس کردن	To prune
تضمین کردن	To guarantee
رضایت بخشی	Satisfiability
بهینه مؤثر	Optimally efficient
رفع تساوی	Tie-breaking
خطر	Risk
تراز هدف	Goal contour
طول راه‌حل	Length of solution
رشد نمایی	Exponential growth
خطا	Error
لگاریتم	Logarithm
رشد کردن	To grow
نقشه رومانی	Map of Romania
آراد	Arad

بهینه را دارد.

ف $\Rightarrow$ E	
In mathematical notation	به زبان ریاضی
Subexponential growth	رشد به صورت نمایی جزئی
Proportional	متناسب
Insistence	پافشاری
Accurate	دقیق
Uninformed search	جستجوی ناآگاهانه
Saving	صرفه جویی
Computation time	زمان محاسبه
Drawback	نقطه ضعف
Memory-bounded heuristic	آروین با حافظه محدود
Iterative deepening	عمیق شونده
Cutoff	تکراری
Unit step cost	هزینه گام واحد
Sorted queue	صف مرتب
Overhead	سربار
Ancestors	اجداد

این موضوع که جستجوی A^* کامل، بهینه و در میان تمامی الگوریتمهای مشابه، بهینه مؤثر می باشد، موضوع رضایت بخشی به حساب می آید. متأسفانه این بدان معنی نیست که A^* پاسخی برای تمامی نیازهای جستجوی ماست. مشکل در اینجا است که در اکثر مسائل، تعداد گرههای فضای جستجوی درون تراز هدف، نسبت به طول راه حل، نمایی می باشد. اگر چه اثبات این نتیجه، خارج از محدوده این کتاب است. نشان می دهیم که اگر خطای تابع آروین سریع تر از لگاریتم هزینه واقعی مسیر رشد کند، رشد نمایی اتفاق خواهد افتاد. در نمادگذاری به زبان ریاضی، شرط رشد به صورت نمایی جزئی این است که

$$|h(n) - h^*(n)| \leq O(\log h^*(n))$$

که در آن $h^*(n)$ هزینه واقعی رسیدن از گره n به هدف است. تقریباً در تمامی آروینهایی که عملاً استفاده می شوند، خطا حداقل متناسب با هزینه مسیر است و رشد نمایی حاصل، نهایتاً هر کامپیوتری را به زانو در می آورد. به همین دلیل، اغلب پافشاری برای یافتن یک راه حل بهینه، عملی نیست. می توان از انواعی از A^* استفاده کرد که راه حلهای نیمه بهینه را به سرعت می یابند یا گاهی اوقات می توان آروینهایی دقیق تر طراحی کرد که کاملاً قابل قبول نباشند. در هر صورت، استفاده از یک آروین خوب در مقایسه با جستجوی ناآگاهانه، باعث صرفه جویی قابل توجهی می شود. در بخش ۴-۲ مسأله طراحی آروینهای خوب را بررسی می کنیم.

زمان محاسبه، نقطه ضعف اصلی A^* نیست. از آنجا که این الگوریتم، تمامی گرههای تولید شده را در حافظه نگهداری می کند (تمامی الگوریتمهای GRAPH-SEARCH این کار را می کنند) معمولاً بسیار بیشتر از آنکه وقت کم بیاورد، حافظه کم می آورد، به همین دلیل A^* در مورد بسیاری از مسائل بزرگ، عملی نیست. الگوریتمهایی که اخیراً به وجود آمده، با هزینه اندک از نظر زمان اجرا، بدون قربانی کردن بهینگی یا کامل بودن، بر مشکل فضا (حافظه) فائق آمده اند. در بخش بعدی آنها را بررسی می کنیم.

جستجوی آروینی با حافظه محدود

ساده ترین راه کاهش احتیاجات حافظه A^* ، پذیرش تفکر عمیق شونده تکراری در جستجوی آروینی است که به الگوریتم A^* عمیق شونده تکراری (IDA^*) منجر می شود. تفاوت اصلی بین IDA^* و الگوریتم عمیق شونده تکراری استاندارد این است که بُرش مورد استفاده، براساس هزینه f (یعنی $g+h$) است نه عمق. در هر تکرار، مقدار برش، کوچک ترین هزینه f میان تمام گرههایی است که از برش تکرار قبلی بیشتر باشند. IDA^* در بسیاری از مسائل که دارای هزینه گام واحد هستند، عملی است و از سربار قابل توجهی که از نگهداری یک صف مرتب از گرهها ناشی می شود اجتناب می کند. متأسفانه، این روش در رابطه با هزینه های واقعی دارای همان مشکلاتی است که نسخه تکرار شونده جستجوی هزینه یکنواخت (تمرین ۳-۱۱) با آنها مواجه بود. این بخش دو الگوریتم جدید با حافظه محدود دیگر که $RBFS$ و MA^* نامیده می شوند را به طور مختصر مورد بررسی قرار می دهد.

جستجوی اول بهترین بازگشتی ($RBFS$) یک الگوریتم بازگشتی ساده است که سعی

می کند عملیات جستجوی اول بهترین استاندارد را تقلید کند با این تفاوت که از فضای خطی استفاده می کند. این الگوریتم در شکل ۴-۵ نشان داده شده است. ساختار آن مشابه جستجوی اول عمق بازگشتی است، اما به جای ادامه مسیر فعلی به طور نامشخص، خود را از مقدار f بهترین مسیر جایگزین قابل دسترس از تمام اجداد گره فعلی مطلع نگه می دارد. اگر گره فعلی از این محدوده فراتر رود، تابع بازگشتی آن را به عقب برمی گرداند تا در یک مسیر جایگزین دیگر قرار گیرد. در حال برگشت به عقب توسط تابع بازگشتی، $RBFS$ مقدار f هر گره در طول مسیر را به بهترین مقدار f فرزندانش تغییر می دهد. بدین طریق، $RBFS$ مقدار f بهترین برگ را در زیردرخت فراموش شده، به خاطر می آورد و در نتیجه، می تواند تعیین کند که آیا در آینده این زیردرخت ارزش توسعه مجدد را دارد یا نه. شکل ۴-۶ نشان می دهد که چگونه $RBFS$ به Bucharest می رسد.

$E \Rightarrow$ ف	
Bucharest	بخارست
Problem	مشکل
Excessive	زیادی
Path	مسیر
Probability	احتمال
To increase	افزایش یافتن
Optimist	خوش بین
Optimal	بهینه
Time complexity	پیچیدگی زمانی
Accuracy	دقت
Exponential increase	افزایش نمایی
Potential	بالقوه
To explore	کاوش کردن
To suffer	رنج بردن
Accessible	قابل دسترسی
Recursive best-first search	جستجوی اول بهترین بازگشتی

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution, or failure
  RBFS(problem, MAKE-NODE(INITIAL-STATE[problem]),  $\infty$ )

function RBFS(problem, node, f-limit) returns a solution, or failure and a new f-cost limit
  if GOAL-TEST[problem](state) then return node
  successors  $\leftarrow$  EXPAND(node, problem)
  if successors is empty then return failure,  $\infty$ 
  for each s in successors do
     $f[s] \leftarrow \max(g(s) + h(s), f[node])$ 
  repeat
    best  $\leftarrow$  the lowest f-value node in successors
    if  $f[best] > f\_limit$  then return failure,  $f[best]$ 
    alternative  $\leftarrow$  the second-lowest f-value among successors
    result,  $f[best] \leftarrow$  RBFS(problem, best,  $\min[f\_limit, alternative]$ )
    if result  $\neq$  failure then return result
```

شکل ۴-۵ الگوریتم جستجوی اول بهترین بازگشتی

$RBFS$ تا حدودی از IDA^* مؤثرتر می باشد ولی باز هم از مشکل تولید مجدد گره های زیادی رنج می برد. در مثال شکل ۴-۶، $RBFS$ ابتدا مسیری را که از Riminu Vilcea می گذرد دنبال می کند. سپس نظرش را عوض کرده و Fagaras را امتحان می کند و دوباره نظرش را عوض می کند. علت این تغییر نظرها این است که هر بار که بهترین مسیر فعلی گسترش می یابد، احتمال زیادی وجود دارد که مقدار f آن افزایش یابد (h معمولاً برای گره های نزدیک تر به هدف، کمتر خوش بین است). وقتی که چنین اتفاقی می افتد، بخصوص در فضا های جستجوی بزرگ ممکن است بهترین مسیر دوم، بهترین مسیر شود. بنابراین جستجو باید به عقب برگردد تا آن را دنبال کند. هر تغییر نظر متناظر است با یک تکرار IDA^* و ممکن است به گسترش مجدد گره های فراموش شده زیادی نیاز داشته باشد تا بهترین مسیر را مجدداً بسازد و آن را یک گره بیشتر گسترش دهد.

همانند A^* ، $RBFS$ نیز یک الگوریتم بهینه است اگر تابع آروین $h(n)$ قابل قبول باشد. پیچیدگی فضایی آن $O(bd)$ است. اما بیان پیچیدگی زمانی آن نسبتاً مشکل است: هم به دقت تابع آروین بستگی دارد و هم به این که در حین گسترش گره ها، بهترین مسیر هر چند وقت یکبار تغییر می کند. هم IDA^* و هم $RBFS$ ذاتاً در معرض افزایش نمایی پیچیدگی ناشی از

سوال امتحان

ف $\Rightarrow$ E

Stage

مرحله

Shortest path

کوتاهترین راه

Limit

محدودیت

Recursive call

فراخوانی

بازگشتی

Current node

گره فعلی

Best

alternative

path

بهترین مسیر

جایگزین

Recursion

بازگشت

Forgotten

subtree

زیردرخت

فراموش شده

To expand

گسترش یافتن

Back up

به عقب برگشتن

Alternative

جایگزین

All available

memory

تمامی حافظه

قابل دسترس

Memory-

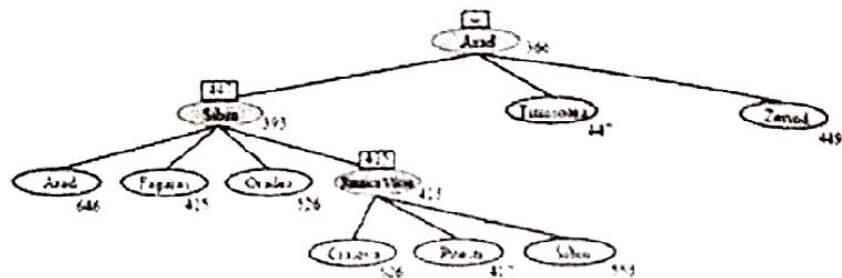
bounded A*

A* مقید به

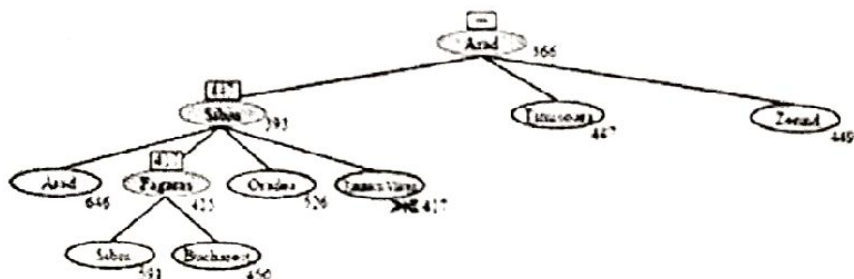
حافظه

جستجوی در گرافها قرار دارند (به بخش ۳-۵ مراجعه کنید) زیرا نمی‌توانند حالت‌های تکراری، به جز آنهایی را که بر روی مسیر فعلی قرار دارند بررسی کنند. در نتیجه، ممکن است یک حالت را چندین بار کاوش کنند.

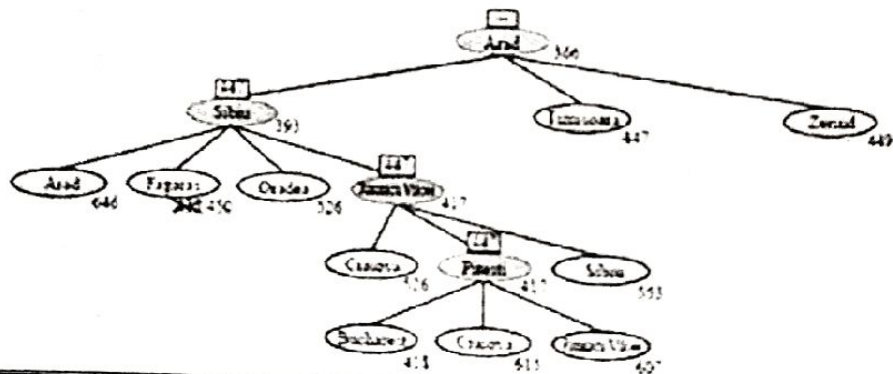
الف) بعد از گسترش *Sibiu Arad* و *Rimnicu Vilcea*



ب) بعد از بازگشت به عقب به *Sibiu* و گسترش *Fagaras*



ج) بعد از تغییر مسیر به *Rimnicu Vilcea* و گسترش *Pitesti*



شکل ۴-۶ مراحل جستجوی *RBFS* برای رسیدن به کوتاه‌ترین راه به *Bucharest*. مقدار محدودیت *f* برای هر فراخوانی بازگشتی در بالای تمام گره‌های فعلی نشان داده شده است. (الف) مسیر از طریق *Rimnicu Vilcea* دنبال می‌شود تا زمانی که بهترین گره فعلی (*Pitesti*) مقداری بدتر از بهترین مسیر جایگزین (*Fagaras*) داشته باشد. (ب) حالت بازگشت باز می‌شود و مقدار بهترین گره زیردرخت فراموش شده (*۴۱۷*) به *Rimnicu Vilcea* برمی‌گردد. سپس، *Fagaras* گسترش می‌یابد و مقدار بهترین گره *۴۵۰* می‌شود. (ج) حالت بازگشتی باز می‌شود و مقدار بهترین گره زیردرخت فراموش شده (*۴۵۰*) به *Fagaras* برمی‌گردد و *Rimnicu Vilcea* گسترش می‌یابد. این بار به این دلیل که بهترین مسیر جایگزین (از طریق *Timisoara*) حداقل هزینه *۴۴۷* دارد، گسترش به سمت *Bucharest* ادامه پیدا می‌کند.

IDA* و **RBFS** از استفاده بسیار اندک از حافظه نیز رنج می‌برند. در خلال تکرارها، **IDA*** فقط یک عدد، یعنی حد مقدار f را نگهداری می‌کند. **RBFS** اطلاعات بیشتری را در حافظه نگهداری می‌کند، هر چند که فقط از $O(bd)$ حافظه استفاده می‌کند؛ حتی اگر حافظه بیشتری در دسترس باشد، **RBFS** امکان استفاده از آن را ندارد.

واضح است که باید از تمامی حافظه قابل دسترس استفاده کنیم. دو الگوریتمی که این کار را می‌کنند **MA*** (مقید به حافظه) و **SMA*** (ساده شده) هستند. ابتدا درباره **SMA*** که ساده‌تر است توضیح داده خواهد شد. **SMA*** دقیقاً مثل **A*** عمل می‌کند، یعنی تا زمانی که حافظه پر شود، بهترین گره را گسترش می‌دهد. در این موقعیت، دیگر نمی‌توان بدون حذف یک گره قدیمی، گره جدیدی به درخت جستجو اضافه کرد. **SMA*** همیشه بدترین گره برگی (گره‌ای با بیشترین مقدار f) را حذف می‌کند. شبیه **RBFS**، **SMA*** نیز مقدار گره فراموش شده را به والدش برمی‌گرداند. بدین ترتیب، جلد یک زیردرخت فراموش شده از کیفیت بهترین مسیر آن زیردرخت آگاه می‌شود. با این اطلاعات **SMA*** زیردرخت را فقط وقتی تولید می‌کند که آشکار شود که تمامی مسیرهای دیگر از مسیری که فراموش کرده است، بدتر هستند. به عبارت دیگر، اگر تمامی زاده‌های یک گره n فراموش شده باشند، آنگاه نمی‌دانیم که از n به کدام سمت برویم، اما این را می‌دانیم که رفتن از n به هرجایی چقدر ارزش دارد.

تولید یک الگوریتم کامل در این مرحله بسیار پیچیده است^۵، اما یک نکته مهم وجود دارد که لازم به ذکر می‌باشد. گفتیم که **SMA*** بهترین گره را گسترش می‌دهد و بدترین گره را حذف می‌کند. اگر تمام گره‌های برگی دارای مقدار f یکسان باشند، چه اتفاقی می‌افتد؟ آنگاه الگوریتم، یک گره را هم برای حذف و هم برای گسترش انتخاب می‌کند. **SMA*** این مشکل را با گسترش جدیدترین گره بهترین و حذف قدیمی‌ترین گره بدترین حل می‌کند. این دو گره تنها در حالتی یکی خواهند بود که فقط یک برگ وجود داشته باشد که در آن حالت، درخت جستجوی فعلی باید یک مسیر از ریشه تا گره باشد که تمامی حافظه را نیز پر کرده است. اگر گره برگی، یک گره هدف نباشد، در آن صورت حتی اگر بر روی یک مسیر راه‌حل بهینه نیز قرار داشته باشد، با این حافظه قابل دسترس، آن راه‌حل نیز دست‌یافتنی نیست. بنابراین، گره می‌تواند دور انداخته شود انگار که هیچ پسینی ندارد.

SMA* کامل است، اگر راه‌حل دست‌یافتنی (یعنی اگر d عمق کم عمیق‌ترین گره هدف، کمتر از اندازه حافظه برحسب گره باشد) وجود داشته باشد. این الگوریتم بهینه است در صورتی که یک راه‌حل بهینه دست‌یافتنی وجود داشته باشد. در غیر این صورت بهترین راه‌حل دست‌یافتنی را برمی‌گرداند. از دیدگاه عملی، **SMA*** می‌تواند الگوریتم همه‌منظوره‌ای برای یافتن راه‌حل‌های بهینه باشد، بخصوص وقتی که فضای حالت یک گراف باشد، هزینه‌های گام یکنواخت نباشند و تولید گره در مقایسه با سربار اضافه نگهداری فهرستهای باز و بسته هزینه‌بر باشد.

در مسائل بسیار سخت، اغلب مواردی پیش می‌آید که **SMA*** مجبور می‌شود بین مجموعه‌ای از مسیرهای نامزد راه‌حل که فقط زیرمجموعه کوچکی از آنها در حافظه جا می‌شود، به طور مدام عقب و جلو برود. (مشابه مسأله دست و پا زدن در سامانه‌های

ف $\rightarrow$ E	
Parent	والد
Ancestor	جد
Subtree	زیردرخت
Forgotten	فراموش شده
Quality	کیفیت
Descendant	زاده
Value	ارزش
Complete	کامل
Complex	پیچیده
Tradeoff	مصلحه
Intractable	مهارشدنی
To remove	حذف کردن
Available memory	حافظه قابل دسترس
To discard	دور انداختن
Overhead	سربار
General-purpose	همه‌منظوره
Uniform	یکنواخت
Thrashing	دست و پا زدن

^۵ یک طرح اولیه از آن در ویرایش اول این کتاب آمده است.

E	ف
Strategy	راهبرد
Fixed	ثابت
Scientist	دانشمند
To design	طراحی کردن
Metalevel state space	فضای حالت فراسطح
Object-level state space	فضای حالت در سطح شیئی
To capture	ثبت کردن
Computation step	گام محاسباتی
Successor	پسین
Metalevel learning	فراگیری فراسطح
unpromising	بدون آتیه
tradeoff	مصالحه
Nature	طبیعت
To slide	لغزاندن
Tile	کاشی
Branching factor	ضریب انشعاب
Horizontal	افقی
Vertical	عمودی

صفحه‌بندی دیسک می‌باشد.) در نتیجه زمان اضافی صرف شده جهت تولید مجدد گره‌های تکراری به این معناست که مسائلی که عملاً در A^* با فرض حافظه نامحدود قابل حل هستند، برای SMA^* مهار نشدنی می‌شوند. به عبارت دیگر، محدودیت‌های حافظه می‌تواند از دیدگاه زمان محاسبه، یک مسأله را مهار نشدنی کنند. هر چند که هیچ نظریه‌ای برای توضیح مصالحه‌ای بین زمان و حافظه وجود ندارد، با این حال به نظر می‌رسد که مشکل اجتناب ناپذیری است. تنها راه گریز، صرف نظر از درخواست بهینگی است.

فراگیری برای جستجوی بهتر

تا اینجا چند راهبرد ثابت (اول سطح، اول بهترین جریصانه و غیره) را معرفی کرده‌ایم که توسط دانشمندان کامپیوتر طراحی شده‌اند. آیا یک کارگزار می‌تواند فرا بگیرد که چگونه بهتر جستجو کند؟ پاسخ مثبت است و این روش وابسته به مفهوم مهمی به نام **فضای حالت فراسطح** می‌باشد. هر حالت در فضای حالت فراسطح، حالت (محاسباتی) داخلی برنامه‌ای را ثبت می‌کند که در یک فضای حالت در سطح شیئی در حال جستجو است، مانند مسأله رومانی. برای مثال، حالت داخلی الگوریتم A^* شامل درخت جستجوی فعلی می‌شود. هر اقدامی در فضای حالت فراسطح، یک گام محاسباتی است که حالت داخلی را تغییر می‌دهد. برای مثال، هر گام محاسباتی در A^* یک گره بزرگی را گسترش می‌دهد و پسین‌های آن را به درخت اضافه می‌کند. در نتیجه، مطابق شکل ۳-۴ رشته درختان جستجو که بزرگ‌تر و بزرگ‌تر می‌شوند را می‌توان به صورت یک مسیر در فضای حالت فراسطح تصور کرد که هر حالت در این مسیر یک درخت جستجو در سطح شیئی می‌باشد.

اکنون مسیر شکل ۳-۴ دارای ۵ گام می‌باشد که یک گام آن، گسترش *Fagaras* است که کمک خاصی به ما نمی‌کند. در مورد مسائل سخت‌تر، از این قبیل گام‌های اشتباه زیاد خواهیم داشت و یک الگوریتم **فراگیری فراسطح** می‌تواند از این تجربیات یاد بگیرد که چگونه از کاوش زیردرخت‌های **بدون آتیه اجتناب** کند. فزونی که برای این نوع یادگیری استفاده می‌شود در فصل ۲۱ توضیح داده خواهد شد. هدف یادگیری، به حداقل رساندن هزینه کلی حل مسأله است (یعنی مصالحه بین هزینه‌های محاسباتی و هزینه مسیر).

۲-۴ توابع آروینی

در این بخش، آروین پازل ۸ تایی را بررسی می‌کنیم تا طبیعت آروین به طور کلی آشکارتر شود. پازل ۸ تایی یکی از اولین مسائل جستجوی آروینی بود. همان طور که در بخش ۳-۲ ذکر گردید، هدف این پازل لغزاندن کاشیها به صورت افقی یا عمودی به داخل فضای خالی است تا زمانی که شکل به دست آمده با شکل هدف منطبق شود (شکل ۴-۷).

هزینه متوسط راه‌حل برای یک نمونه پازل ۸ تایی که به صورت تصادفی تولید شده باشد در حدود ۲۲ گام است. ضریب انشعاب در حدود ۳ است. (وقتی که کاشی خالی در وسط قرار دارد، چهار حرکت امکان‌پذیر است. وقتی که در گوشه باشد، دو حرکت و وقتی که کنار باشد، سه حرکت ممکن است.) این بدان معنی است که یک جستجوی کامل به عمق ۲۲ تقریباً

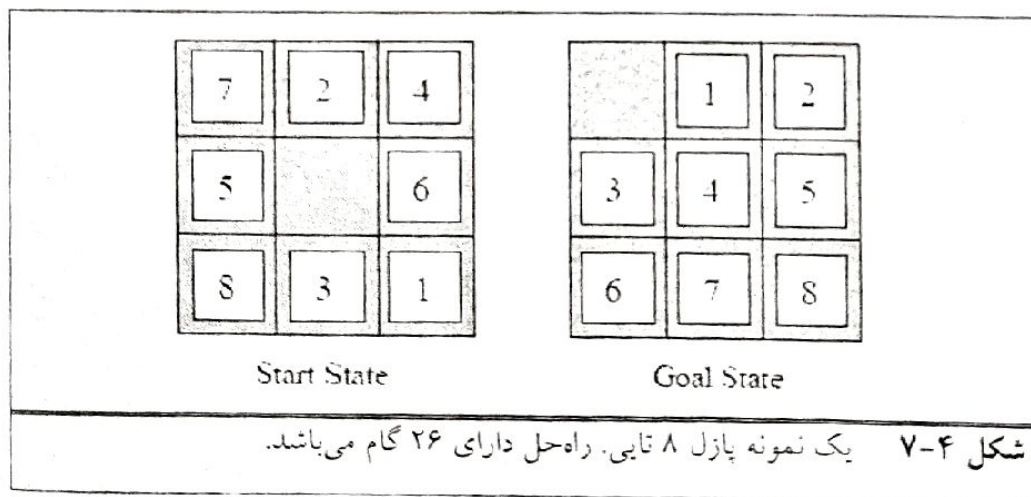
$10^{11} \approx 3/1 \approx 3^{11}$ حالت را بررسی می‌کند. با نگهداری سابقه حالت‌های تکراری، می‌توانیم این عدد را با ضریبی حدود ۱۷۰۰۰۰ کاهش دهیم، زیرا فقط $9!/2 = 18440$ حالت متمایز قابل دست‌یابی وجود دارد. (تمرین ۳-۴ را نگاه کنید.) این تعداد قابل پایش است، اما تعداد متناظر در مورد یک پازل ۱۵ تایی حدوداً 10^{13} است. بنابراین، گام بعدی یافتن یک تابع آروین خوب است. اگر بخواهیم کوتاه‌ترین راه‌حل را با استفاده از A^* پیدا کنیم، به یک تابع آروین نیاز داریم که هرگز تعداد گامها تا هدف را بیش از اندازه برآورد نکند. تاریخچه طولانی از چنین آروینهایی برای پازل ۱۵ تایی وجود دارد. در اینجا دو نمونه معمول را بیان می‌کنیم:

• $h_1 =$ تعداد کاشیهایی که در جای خود قرار نگرفته‌اند. در شکل ۴-۷ تمامی هشت کاشی خارج از جای خود قرار دارند. بنابراین، در حالت شروع، $h_1 = 8$ خواهد بود. h_1 یک آروین قابل قبول است، زیرا واضح است که هر کاشی که خارج از مکانش قرار دارد باید حداقل یک‌بار جابه‌جا شود.

• $h_2 =$ مجموع فاصله کاشیها از موقعیت‌های هدفشان. از آنجا که کاشیها به طور قطری نمی‌توانند حرکت کنند، مجموع فواصل افقی و عمودی را محاسبه می‌کنیم. گاهی اوقات این فاصله را فاصله شهر قالبی یا فاصله منتهن می‌نامند. h_2 نیز قابل قبول است زیرا تنها کاری که هر حرکت می‌تواند انجام دهد جابه‌جایی یک کاشی به اندازه یک قدم نزدیک‌تر به هدف است. فاصله منتهن کاشیهای ۱ تا ۸ در حالت شروع، به قرار زیر است:

$$h_2 = 3+1+2+2+2+3+3+2=18$$

همان طور که امیدوار بودیم، هیچ کدام از این دو هزینه واقعی راه‌حل را که ۲۶ است بیش از اندازه برآورد نمی‌کنند.



تأثیر دقت آروین بر کارایی

ضریب مؤثر انشعاب، b^* ، روشی برای بیان کیفیت یک آروینی است. اگر تعداد کل گرههایی که توسط A^* برای یک مسئله خاص تولید می‌شود، N و عمق راه‌حل d باشد، آنگاه b^* ضریب انشعابی است که یک درخت یکنواخت به عمق d باید داشته باشد تا دارای $N+1$ گره

$E \Rightarrow$	ف
To be misplaced	در جای اشتباه
	قرار گرفتن
Tile	کاشی
To be moved	جابه‌جا شدن
Distance	فاصله
Diagonal	قطری
City block distance	فاصله شهر قالبی
Manhattan distance	فاصله منتهن
Move	حرکت
To overestimate	بیش از اندازه برآورد کردن
Efficiency	کارایی
Effect	تأثیر
Accuracy	دقت
Effective branching factor	ضریب مؤثر انشعاب
Uniform tree	درخت یکنواخت
Constant	ثابت
Usefulness	سودمندی

باشد. بنابراین:

ف $\Rightarrow$ E

Test

آزمایش

$$N+1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

Random problem

مسأله تصادفی

Iterative deepening search

جستجوی

عمیق شونده

تکراری

Strategy

راهبرد

Effective branching factor

ضریب مؤثر

انشعاب

Uninformed

ناآگاهانه

Comparison

مقایسه

Heuristic

آروین

Node

گره

To dominate

برتری داشتن

Domination

برتری

Efficiency

کارایی

At least

حداقل

Surely

یقیناً

To expand

گسترش دادن

Except

به استثنای

برای مثال، اگر A^* راه حلی را در عمق ۵ و با استفاده از ۵۲ گره بیابد، آنگاه ضریب مؤثر انشعاب $1/92$ خواهد بود. ضریب مؤثر انشعاب می تواند در نمونه های مختلف یک مسأله متفاوت باشد، ولی معمولاً برای مسائلی که به اندازه کافی سخت هستند، نسبتاً ثابت است. در نتیجه، اندازه گیری تجربی b^* مجموعه کوچکی از مسائل می تواند راهنمای خوبی در مورد سودمندی کلی جستجوی آروینی باشد. یک آروین که به خوبی طراحی شده باشد، دارای مقدار b^* نزدیک به ۱ می باشد که حل مسائل نسبتاً بزرگ را ممکن می سازد.

برای آزمایش توابع آروین h_1 و h_2 ۱۲۰۰ مسأله تصادفی با طول راه حل ۲ تا ۲۴ تولید کردیم (۱۰۰ تا برای اعداد زوج) و آنها را با جستجوی عمیق شونده تکراری (IDS) و جستجوی درخت A^* با استفاده از هر دوی h_1 و h_2 حل کردیم. شکل ۴-۸ تعداد متوسط گره های گسترش یافته و ضریب مؤثر انشعاب هر راهبرد را نشان می دهد. این نتایج نشان می دهد که h_2 از h_1 بهتر و A^* از IDS بسیار بهتر است. در راه حل هایی به طول ۱۴، A^* همراه با h_2 ۳۰۰۰۰ بار از IDS ناآگاهانه مؤثرتر است.

d	هزینه جستجو			ضرایب مؤثر انشعاب		
	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	364403	227	73	2.78	1.42	1.24
14	5	539	113	-	1.44	1.23
16	-	1301	211	-	1.45	1.25
18	-	3056	363	-	1.46	1.26
20	-	7276	676	-	1.47	1.27
22	-	18094	1219	-	1.48	1.28
24	-	39135	1641	-	1.48	1.26

شکل ۴-۸ مقایسه هزینه های جستجو و ضرایب مؤثر انشعاب در مورد الگوریتم های IDS و A^* با h_1 ، h_2 داده ها متوسط ۱۰۰ نمونه از پازل ۸ تایی، با راه حل هایی با طول مختلف هستند.

ممکن است این سؤال برایتان پیش آید که آیا h_2 همیشه از h_1 بهتر است. پاسخ مثبت است. به سادگی از تعاریف این دو آروین مشخص می شود که برای هر گره n ، $h_2(n) \geq h_1(n)$ است. بنابراین، می گوئیم که h_2 بر h_1 برتری دارد. برتری را مستقیماً می توان به کارایی تعبیر کرد: A^* با استفاده از h_2 هرگز گره های بیشتری نسبت به A^* با استفاده از h_1 گسترش نخواهد داد (به استثنای بعضی گره ها که برای آنها $f(n) = C^*$ برقرار است). بحث ساده است. اظهار نظر

صفحه ۱۱۹ را به خاطر آورید که تمام گره‌های با $C^* < f(n)$ یقیناً گسترش خواهد یافت. ولی از آنجا که h_2 برای تمامی گره‌ها حداقل به بزرگی h_1 است هر گرهی که یقیناً توسط جستجوی A^* با h_2 گسترش می‌یابد، یقیناً با h_1 نیز گسترش می‌یابد و h_1 همچنین ممکن است باعث گسترش گره‌های دیگری هم بشود. بنابراین، همیشه بهتر است که از تابع آروینی با مقادیر بالاتر استفاده کنیم. به شرطی که بیش از اندازه تخمین نزنند و زمان محاسبات برای آروین بیش از اندازه بزرگ نباشد.

ابداع توابع آروین قابل قبول

تاکنون دیده‌ایم که هم h_1 (کاشیهایی که در جای خود قرار ندارند) و هم h_2 (فاصله منتهن) آروینهای نسبتاً خوبی برای پازل ۸ تایی هستند و از میان این دو، h_2 بهتر است. چگونه می‌توان h_2 را پیدا کرد؟ آیا یک کامپیوتر می‌تواند چنین آروینی را به طور مکانیکی ابداع کند؟

h_1 و h_2 برآوردهایی از طول مسیر باقیمانده برای پازل ۸ تایی هستند، ولی این توابع برای نسخه‌های ساده شده این پازل طول مسیرها را کاملاً دقیق ارائه می‌کنند. اگر قواعد پازل تغییر می‌کرد به نحوی که یک کاشی می‌توانست به جای این که فقط به مربع خالی مجاور حرکت کند، به هر جایی که می‌خواست حرکت می‌کرد، در آن صورت h_1 تعداد دقیق گامها در کوتاه‌ترین راه‌حل را ارائه می‌کرد. به طور مشابه، اگر یک کاشی می‌توانست به اندازه یک مربع به هر سمتی، حتی به درون یک مربع اشغال‌شده، حرکت کند، در آن صورت h_2 تعداد دقیق گامها در کوتاه‌ترین راه‌حل را ارائه می‌کرد. (مسئله‌ای که تعداد کمتری محدودیت برای اقدامات دارد، یک مسئله تعدیل‌شده نامیده می‌شود. هزینه یک راه‌حل بهینه برای یک مسئله تعدیل‌شده، یک آروین قابل قبول برای مسئله اصلی است. این آروین قابل قبول است زیرا راه‌حل بهینه در مسئله اصلی، بنا بر تعریف، راه‌حلی در مسئله تعدیل‌شده است) و در نتیجه، باید از نظر هزینه با هزینه راه‌حل بهینه در مسئله تعدیل‌شده یکی باشد. از آنجا که آروین به دست آمده، یک هزینه دقیق برای مسئله تعدیل‌شده می‌باشد، باید از نامساوی مثلث تبعیت کند و در نتیجه، سازگار است) (به صفحه ۱۱۷ مراجعه کنید).

اگر تعریف یک مسئله به زبان رسمی نوشته شده باشد، ساخت خودکار مسائل تعدیل‌شده امکان‌پذیر است.^۶ برای مثال، اگر اقدامات پازل ۸ تایی به صورت ذیل توصیف شده باشند:

یک کاشی می‌تواند از مربع A به مربع B حرکت کند اگر A به طور افقی یا به طور عمودی مجاور B باشد و B خالی باشد.

می‌توان با حذف یک یا هر دو شرط، سه مسئله تعدیل‌شده تولید کرد:

(الف) یک کاشی می‌تواند از مربع A به مربع B حرکت کند اگر A مجاور B باشد.

(ب) یک کاشی می‌تواند از مربع A به مربع B حرکت کند اگر B خالی باشد.

(ج) یک کاشی می‌تواند از مربع A به مربع B حرکت کند.

$E \rightarrow$	ف
To invent	ابداع کردن
Admissible	قابل قبول
Mechanical	مکانیکی
Estimate	برآورد
Rule	قاعده
To be changed	تغییر کردن
Adjacent	مجاور
Occupied	اشغال شده
Relaxed problem	مسئله تعدیل‌شده
Original problem	مسئله اصلی
Triangle inequality	نامساوی مثلث
Consistent	سازگار
Definition	تعریف
Formal	رسمی
To remove	حذف کردن
Condition	شرط
Subproblem	زیرمسئله

^۶ در فصلهای ۸ و ۱۱ زبانهای رسمی مناسب این کار را توضیح می‌دهیم. با توضیحات رسمی که قابل به‌کارگیری باشند، ساخت مسائل تعدیل‌شده می‌تواند خودکار شود. در حال حاضر از فارسی (انگلیسی) استفاده می‌کنیم.

$E \Rightarrow$	ف
Corresponding	متناظر
Rubik's cube	مکعب روبیک
Set	مجموعه
Composite	مرکب
Function	تابع
The node in question	گره مورد نظر
Accurate	دقیق
Partial	جزئی
Domination	برتری
Instance	نمونه
Obvious	واضح
Complete problem	مسئله کامل
8-puzzle	پازل ۸ تایی
Perfect heuristic	آرئین جامع
Breadth-first search	جستجوی اول سطح
Tradeoff	مصلحه

از (الف) می‌توانیم h (فاصله منتهن) را استخراج کنیم. استدلال ما این است که اگر هر کاشی را به نوبت به سمت مقصدش حرکت دهیم، h امتیاز مناسبی خواهد داشت. آرئینی که از (ب) مشتق می‌شود در تمرین ۴-۹ بحث می‌شود. از (ج) می‌توانیم h_1 (کاشیهای ناب‌جا) را استخراج کنیم، زیرا اگر کاشیها می‌توانستند با یک گام به مقصد مورد نظرشان بروند جواب h_1 صحیح می‌بود. توجه کنید که این مسئله حیاتی است که مسائل تعدیل‌شده تولیدشده توسط این فن بتوانند اساساً بدون جستجو حل شوند، زیرا قواعد تعدیل‌شده اجازه می‌دهند که مسئله به هشت زیرمسئله مستقل تجزیه شود. اگر حل مسئله تعدیل‌شده سخت باشد، آنگاه به‌دست آوردن مقادیر آرئینهای متناظر پرهزینه خواهد بود.^۷

برنامه‌ای به نام ABSOLVER می‌تواند با استفاده از روش "مسئله تعدیل‌شده" و فنون مختلف دیگر (Prieditis, 1993) به طور خودکار از تعاریف مسئله، آرئینها را تولید کند. ABSOLVER یک آرئین جدید برای پازل ۸ تایی تولید کرد که بهتر از تمامی آرئینهای موجود بود و اولین آرئین قابل استفاده را برای پازل معروف مکعب روبیک پیدا کرد.

مشکل تولید توابع آرئین جدید این است که معمولاً نمی‌توان آرئینی را که مشخصاً بهترین باشد پیدا کرد. اگر مجموعه‌ای از آرئینهای قابل قبول $h_1, \dots, h_m$ برای یک مسئله موجود باشد و هیچ کدام از آنها بر دیگری برتری نداشته باشد، کدامیک را باید انتخاب کنیم؟ همان طور که مشخص است باید گزینش انجام دهیم. با تعریف ذیل می‌توانیم بهترینها را داشته باشیم:

$$h(n) = \max \{h_1(n), \dots, h_m(n)\}$$

این آرئین مرکب، از تابعی استفاده می‌کند که در گره مورد نظر دقیق‌ترین است. از آنجا که آرئینهای جزئی قابل قبول هستند، h قابل قبول است. به سادگی می‌توان ثابت کرد که h سازگار نیز هست. علاوه بر این، h بر تمامی آرئینهای جزئی‌اش برتری دارد.

آرئینهای قابل قبول را همچنین می‌توان از هزینه راه‌حل یک زیرمسئله از یک مسئله خاص به‌دست آورد. برای مثال، شکل ۴-۹ یک زیرمسئله از نمونه پازل ۸ تایی شکل ۴-۷ را نشان می‌دهد. این زیرمسئله شامل رساندن کاشیهای ۱ و ۲ و ۳ و ۴ به محل‌های صحیح‌شان است. واضح است که هزینه راه‌حل بهینه این زیرمسئله، یک حد پایین برای هزینه مسئله کامل است. در بعضی موارد، این روش خیلی دقیق‌تر از فاصله منتهن می‌باشد.

تفکر اصلی پایگاههای اطلاعاتی الگو، ذخیره هزینه دقیق راه‌حل‌های نمونه‌هایی از زیرمسئله‌های ممکن (در مثال ما، هر چیدمان ممکن برای ۴ کاشی و مربع خالی) است. (دقت کنید که موقعیت چهار کاشی دیگر ارتباطی به هدف حل این زیرمسئله ندارد، ولی حرکت این کاشیها در هزینه تأثیر می‌گذارد.) سپس، برای حالت‌های کاملی که در طول جستجو با آن برخورد کردیم، با یافتن چیدمان زیرمسئله متناظر در پایگاه اطلاعاتی، آرئین قابل قبول h_{1111} را محاسبه می‌کنیم. خود پایگاه اطلاعاتی با جستجوی رو به عقب از حالت هدف و ثبت هزینه هر الگوی جدیدی که با آن برخورد می‌کند ساخته می‌شود. هزینه این جستجو توسط نمونه‌های متعدد

^۷ توجه داشته باشید که می‌توان در واقع یک آرئین جامع را با دادن اجازه به h برای انجام یک جستجوی کامل اول سطح "به صورت پنهانی" به دست آورد. بنابراین، یک مصالحه بین دقت و زمان محاسبه برای توابع آرئین وجود دارد.

مسائل متوالی مستهلک می‌شود.

ف E

Pattern
database

پایگاه اطلاعاتی

الگو

Subproblem

زیرمسئله

Backward
search

جستجوی رو به

عقب

To be
amortized

مستهلک شدن

Arbitrary

اختیاری

Maximum

بیشینه

Composite
heuristic

آروین مرکب

Random

تصادفی

Factor

ضریب

Overlap

همپوشانی

To be added

جمع شدن

Shared

مشترک

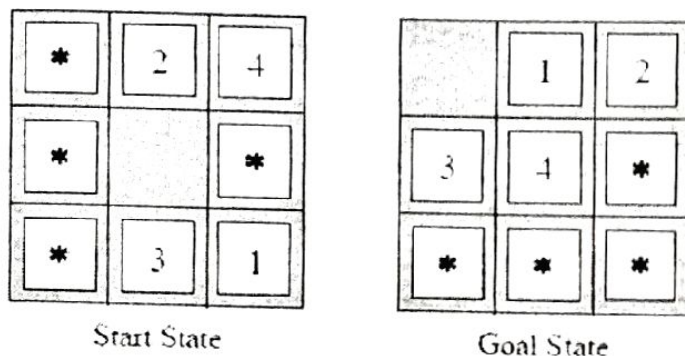
Unlikely

غیرمتحمل

Disjoint pattern
database

پایگاه اطلاعاتی

پراکنده الگو



Start State

Goal State

شکل ۴-۹ یک زیرمسئله از نمونه پازل ۸ تایی که در شکل ۴-۷ آمده است. در اینجا، هدف رساندن کاشیهای ۱ و ۲ و ۳ و ۴ به محل صحیح‌شان است، بدون این که نگران سرنوشت کاشیهای دیگر باشیم.

انتخاب ۱-۲-۳-۴ کاملاً اختیاری است. می‌توان پایگاههای اطلاعاتی برای ۵-۶-۷-۸ و برای ۲-۳-۴-۸ و ... ساخت. هر یک از این پایگاههای اطلاعاتی یک آروین قابل قبول تولید می‌کنند و این آروینها همان‌طور که قبلاً توضیح داده شد، می‌توانند با برداشتن مقدار بیشینه با هم ترکیب شوند. چنین آروین مرکبی، بسیار دقیق‌تر از فاصله منتهن است. تعداد گرههای تولید شده در هنگام حل پازل ۱۵ تایی تصادفی، می‌تواند با ضریب ۱۰۰۰ کاهش یابد.

ممکن است از خود پرسید آیا آروینهایی که از پایگاه اطلاعاتی ۱-۲-۳-۴ و ۵-۶-۷-۸ به دست می‌آیند، از آنجا که دو زیرمسئله بدون همپوشانی به نظر می‌رسند، می‌توانند با هم جمع شوند؟ آیا باز هم این یک آروین قابل قبول خواهد بود؟ پاسخ منفی است، زیرا راه‌حلهای زیرمسئله ۱-۲-۳-۴ و زیرمسئله ۵-۶-۷-۸ برای یک حالت خاص تقریباً به طور قطعی چند حرکت مشترک (امکان ندارد که ۱-۲-۳-۴ بتواند به محل خودش حرکت کند، بدون این که دست به ۵-۶-۷-۸ بزند و برعکس) دارند. اما اگر این حرکتها را بشماریم چطور؟ یعنی به جای کل هزینه حل زیرمسئله ۱-۲-۳-۴ فقط تعداد حرکتهایی را که ۱-۲-۳-۴ شامل می‌شود، ثبت کنیم. پس به سادگی می‌توان دید که مجموع این دو هزینه هنوز حد پایین هزینه حل مسئله کامل است. این موضوع، ایده اصلی پایگاههای اطلاعاتی پراکنده الگو را تشکیل می‌دهد. با استفاده از چنین پایگاههای اطلاعاتی، حل پازل ۱۵ تایی تصادفی در چند میلی‌ثانیه امکان‌پذیر می‌شود (تعداد گرههای تولیدشده در مقایسه با استفاده از فاصله منتهن با ضریب ۱۰۰۰۰ کاهش می‌یابد). در مورد پازل ۲۴ تایی سرعت در حدود یک میلیون برابر افزایش می‌یابد.

پایگاههای اطلاعاتی پراکنده الگو برای پازلها با کاشی لغزان قابل استفاده هستند، زیرا این مسئله را می‌توان به نحوی تقسیم کرد که هر حرکت تنها یک زیرمسئله را تحت تأثیر قرار دهد (زیرا در هر لحظه فقط یک کاشی حرکت می‌کند). در مورد مسئله‌ای مانند مکعب روییک این نوع تقسیم قابل اجرا نیست زیرا هر حرکت ۸ یا ۹ مکعب از ۲۶ مکعب را تحت تأثیر قرار می‌دهد. در حال حاضر مشخص نیست که برای چنین مسائلی چگونه پایگاههای

اطلاعاتی پراکنده را باید تعریف کنیم.

فراگیری تجربی آروینها

ف $E \Rightarrow$

Relaxed

تعدیل شده

To design

طراحی کردن

Optimal solution

راه حل بهینه

Experience

تجربه

Example

مثال

Inductive learning

فراگیری

استقرایی

Technique

فن

Neural network

شبکه عصبی

Decision tree

درخت

تصمیم گیری

Reinforcement learning

فراگیری تقویتی

Feature

ویژگی

Linear

خطی

Layout

چیدمان

To explore

کاوش کردن

Systematic

بسامان

To predict

پیش بینی کردن

Final

نهایی

Positive

مثبت

Negative

منفی

انتظار می رود تابع آروین $h(n)$ هزینه راه حل را با شروع از گره n برآورد کند. چگونه یک کارگزار می تواند چنین تابعی بسازد؟ در بخش قبلی یک راه حل ارائه شد (بدین ترتیب که مسائل تعدیل شده ای طراحی کنیم که برای آنها به آسانی بتوان یک راه حل بهینه پیدا کرد). راه حل دیگر فراگیری از طریق تجربه است. منظور از "تجربه" در اینجا به عنوان مثال تعداد زیادی پازل ۸ تایی است. هر راه حل بهینه در مسأله پازل ۸ تایی مثالی است که می توان از آنها $h(n)$ را فرا گرفت. هر مثال شامل یک حالت از مسیر راه حل و هزینه واقعی راه حل از آن نقطه می شود. از این مثالها می توان با استفاده از یک الگوریتم فراگیری استقرایی تابع $h(n)$ را ساخت که (با کمی شانس) می تواند هزینه های راه حل را برای حالت های دیگری که در طول جستجو پیش می آیند، پیش بینی کند. فنون انجام این کار با استفاده از شبکه های عصبی، درخت های تصمیم گیری و روش های دیگر در فصل ۱۸ نشان داده شده اند. (روش های فراگیری تقویتی که در فصل ۲۱ توضیح داده می شوند نیز قابل استفاده هستند).

هنگامی که روش های فراگیری استقرایی را به جای فقط توصیفاتی خام از حالت، با ویژگی هایی از حالت که مربوط به ارزش آن است تغذیه کنیم، از همیشه بهتر عمل می کنند. برای مثال، ویژگی "تعداد کاشیهایی که در جای خود قرار ندارند" ممکن است در پیش بینی فاصله واقعی یک حالت از هدف، سودمند باشد. این ویژگی را $x_1(n)$ می نامیم. می توانیم ۱۰۰ چیدمان پازل ۸ تایی که به صورت تصادفی تولید شده را در نظر بگیریم و آماری از هزینه های واقعی راه حل آنها گردآوری نماییم. ممکن است دریابیم که وقتی $x_1(n)$ دارای مقدار ۵ است، هزینه متوسط راه حل حدود ۱۴ است و غیره. با در دست داشتن این داده ها از مقدار x_1 می توان برای پیش بینی $h(n)$ استفاده کرد. بدیهی است که می توان از چند ویژگی استفاده کرد. ویژگی دومی مانند $x_2(n)$ می تواند "تعداد زوج کاشیهایی مجاور که در حالت هدف نیز مجاور هستند" باشد. چگونه باید $x_1(n)$ و $x_2(n)$ را ترکیب کرد تا $h(n)$ را پیش بینی کنیم؟ یک رویکرد معمول استفاده از یک ترکیب خطی است:

$$h(n) = c_1 x_1(n) + c_2 x_2(n)$$

ثابت های c_1 و c_2 به نحوی تنظیم می شوند که بیشترین تطبیق با داده های واقعی هزینه های راه حل را داشته باشند. احتمالاً c_1 مثبت و c_2 منفی خواهد بود.

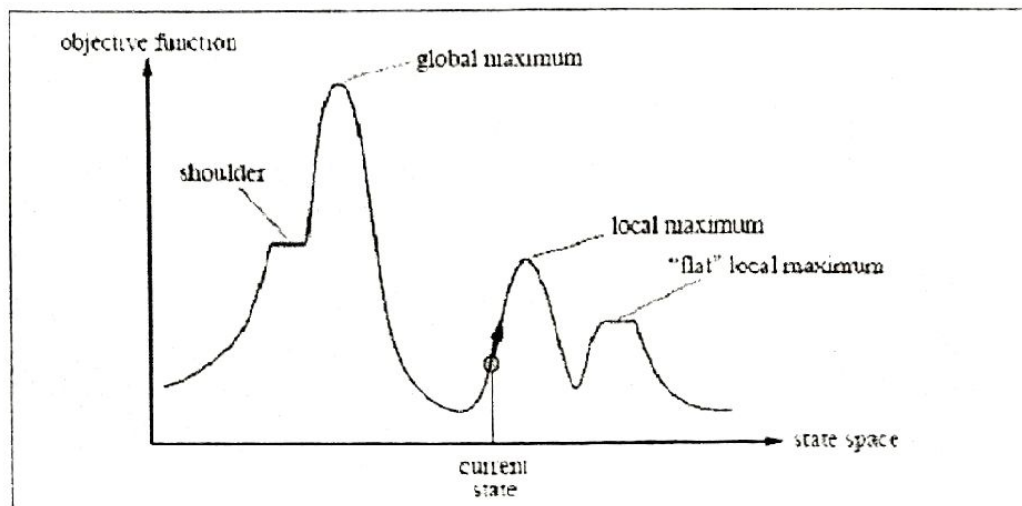
۳-۴ الگوریتم های جستجوی محلی و مسائل بهینه سازی

الگوریتم های جستجویی که تا به حال دیده ایم، برای کاوش بسامان فضای جستجو طراحی شده اند. این کار با نگهداری یک یا چند مسیر در حافظه و ثبت این که کدام گزینه ها در هر نقطه در طول مسیر کاوش شده اند و کدامها کاوش نشده اند، میسر می شود. هنگامی که یک هدف پیدا می شود، مسیر تا هدف نیز راه حل مسأله را تشکیل می دهد.

با وجود این در بسیاری از مسائل، مسیر به هدف ارتباطی ندارد. برای مثال، در مسئله ۸ وزیر (به صفحه ۷۹ مراجعه نمایید)، چیدمان نهایی وزیرها اهمیت دارد، نه ترتیبی که آنها اضافه می‌شوند. این نوع مسائل، کاربردهای مهم زیادی از قبیل طراحی مدارهای مجتمع، چیدمان کف کارخانه (چیدمان دستگاههای کارخانه)، زمان‌بندی کاری، برنامه‌نویسی خودکار، بهینه‌سازی شبکه مخابرات راه دور، مسیریابی خودرو و مدیریت سبد سهام را در برمی‌گیرد.

اگر مسیر هدف اهمیت نداشته باشد، می‌توان دسته‌های متعددی از الگوریتمها را در نظر گرفت که اصلاً به مسیر اهمیتی نمی‌دهند. الگوریتمهای جستجوی محلی، به جای مسیرهای چندگانه تنها با استفاده از حالت فعلی عمل می‌کنند و عموماً فقط به همسایگان آن حالت، تغییر مکان می‌دهند. معمولاً مسیریابی که توسط جستجو دنبال می‌شوند نگهداری نمی‌شوند. اگرچه الگوریتمهای جستجوی محلی بسامان نیستند، ولی دو مزیت کلیدی دارند: (۱) از حافظه بسیار کم استفاده می‌کنند (معمولاً یک مقدار ثابت) و (۲) اغلب می‌توانند در فضاهای حالت بزرگ یا نامتناهی (پیوسته) که برای آنها الگوریتمهای بسامان مناسب نیستند، راه‌حلهای معقولی بیابند.

الگوریتمهای جستجوی محلی علاوه بر یافتن اهداف، در حل مسائل محض بهینه‌سازی سودمند می‌باشند که در آنها هدف، یافتن بهترین حالت براساس تابع هدف است. بسیاری از مسائل بهینه‌سازی با مدل جستجوی "استاندارد" که در فصل ۳ بیان شد، مطابقت ندارند. برای مثال در طبیعت یک تابع هدف (آمادگی تولید مثل) وجود دارد که براساس نظریه تکامل داروینی می‌توان فرض کرد که برای بهینه‌سازی آن تلاش می‌کند، اما برای این مسئله هیچ آزمون هدف و "هزینه مسیری" وجود ندارد.



شکل ۴-۱۰ دورنمایی از فضای حالت یک‌بعدی که در آن ارتفاع، متناظر با تابع هدف است. هدف یافتن بیشینه سراسری است. جستجوی تپه‌نوردی، با تغییر حالت فعلی آن را بهبود می‌بخشد (در شکل با پیکان نشان داده می‌شود). در متن، انواع ویژگیهای نقشه‌برداری تعریف شده‌اند.

برای درک جستجوی محلی، توجه به دورنمای فضای حالت (مانند شکل ۴-۱۰) بسیار مفید

 ف $\Rightarrow$ E

Integrated circuits

مدارهای مجتمع

Factory-floor layout

چیدمان کف

کارخانه

Job-shop scheduling

زمان‌بندی کاری

Automatic programming

برنامه‌نویسی

خودکار

Optimization

بهینه‌سازی

Vehicle routing

مسیریابی خودرو

Portfolio

سبد سهام

Local search

جستجوی محلی

Infinite

نامتناهی

Reasonable

معقول

Objective function

تابع هدف

One-dimensional

یک‌بعدی

Elevation

ارتفاع

Hill-climbing

تپه‌نوردی

State space landscape

دورنمای فضای

حالت

ف E	شامل
Peak	قله
Global maximum	بیشینه سراسری
Negative sign	علامت منفی
To explore	کاوش کردن
Complete	کامل
Optimal	بهینه
Hill-climbing search	جستجوی تپهنوردی
Loop	حلقه
Neighbor	همسایه
Global Minimum	کمینه سراسری
Data structure	ساختمان داده
Complete-state formulation	تدوین حالت کامل
Mount Everest	کوه اورست
Steepest ascent version	نسخه تیزترین صعود

خواهد بود. یک دورنما هم شامل "محل" (که توسط حالت تعریف می‌شود) و هم شامل "ارتفاع" (که توسط مقدار تابع هزینه آروین یا تابع هدف تعریف می‌شود) می‌باشد. اگر ارتفاع متناظر با هزینه باشد، آنگاه هدف، یافتن عمیق‌ترین دره (یک کمینه سراسری) است و اگر ارتفاع متناظر با تابع هدف باشد آنگاه هدف، یافتن بلندترین قله (یک بیشینه سراسری) است. (فقط با افزودن یک علامت منفی، می‌توان یکی را به دیگری تبدیل کرد.) الگوریتمهای جستجوی محلی، این دورنما را کاوش می‌کنند. یک الگوریتم جستجوی محلی کامل، همیشه هدف را (در صورت وجود) پیدا می‌کند و یک الگوریتم بهینه همیشه یک کمینه یا بیشینه سراسری را پیدا می‌کند.

جستجوی تپهنوردی

الگوریتم جستجوی تپهنوردی در شکل ۴-۱۱ نشان داده شده است. این الگوریتم فقط از یک حلقه تشکیل می‌شود که مدام در جهت افزایش مقدار، حرکت می‌کند (یعنی به سمت بالای تپه). الگوریتم هنگامی خاتمه پیدا می‌کند که به قله‌ای برسد که در آنجا هیچ همسایه‌ای، مقدار بیشتری نداشته باشد. در این الگوریتم، درخت جستجو را نگهداری نمی‌شود. بنابراین، در ساختمان داده گره فعلی تنها نیاز به ثبت حالت و مقدار تابع هدف آن می‌باشد. جستجوی تپهنوردی، فراتر از همسایه‌های مجاور حالت فعلی را نگاه نمی‌کند. این موضوع، مثل تلاش برای یافتن قله کوه اورست در مه غلیظ است در حالی که از بیماری فراموشی نیز رنج می‌بریم.

function HILL-CLIMBING(*problem*) returns a state that is a local maximum
inputs: *problem*, a problem
local variables: *current*, a node
neighbor, a node

current ← MAKE-NODE(INITIAL-STATE[*problem*])
loop do
neighbor ← a highest-valued successor of *current*
if VALUE[*neighbor*] ≤ VALUE[*current*] **then return** STATE[*current*]
current ← *neighbor*

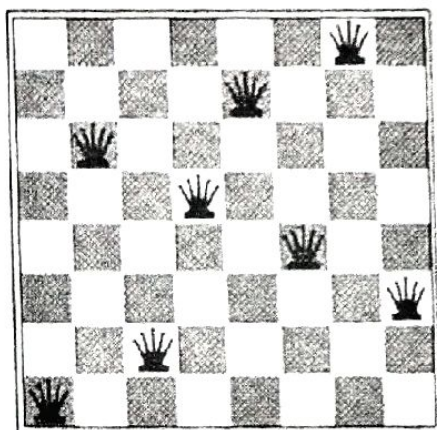
شکل ۴-۱۱ الگوریتم جستجوی تپهنوردی (نسخه تیزترین صعود) که پایه‌ای‌ترین فن جستجوی محلی است. در هر گام بهترین همسایه جایگزین گره فعلی می‌شود. در این نسخه، بهترین همسایه، همسایه‌ای است که بیشترین VALUE را دارد، اما اگر از برآورد آروین هزینه h استفاده می‌کردیم، همسایه‌ای پیدا می‌شد که کمترین h را دارد.

با یک مثال از مسئله ۸ وزیر که در صفحه ۷۹ معرفی گردید، جستجوی تپهنوردی را روشن می‌کنیم. الگوریتمهای جستجوی محلی معمولاً از تدوین حالت کامل که در آن هر حالت شامل ۸ وزیر، یعنی یکی در هر ستون، استفاده می‌کنند. تابع پسین تمامی حالت‌های ممکن را که با جابه‌جایی یک وزیر به مربع دیگری در همان ستون تولید می‌شود، برمی‌گرداند (بنابراین، هر حالت $8 \times 7 = 56$ پسین دارد).
 تابع هزینه آروین h تعداد جفت وزیرهایی است که به صورت مستقیم و یا غیر مستقیم

در حال حمله به یکدیگر هستند. کمینه سراسری این تابع صفر است که فقط در راه حل کامل اتفاق می افتد. شکل ۴-۱۲-الف حالتی را با $h=17$ نشان می دهد. این شکل همچنین، مقادیر تمامی پسینهای آن را نیز نشان می دهد که بهترین پسین دارای $h=12$ است. در صورتی که در الگوریتمهای تپهنوردی بیش از یک بهترین پسین وجود داشته باشد معمولاً از مجموعه بهترین پسینها یکی را به صورت تصادفی انتخاب می کنند.

تپهنوردی را گاهی جستجوی حریصانه محلی می نامند زیرا یک حالت همسایه خوب را بدون این که از قبل فکر کند که از آنجا به کجا خواهد رفت، انتخاب می کند. با وجود این که حرص یکی از هفت گناه کبیره به شمار می رود، الگوریتمهای حریصانه اغلب بسیار خوب عمل می کنند. تپهنوردی اغلب پیشرفت بسیار سریعی به سمت راه حل دارد، زیرا معمولاً بهبود یک حالت بد خیلی ساده است. برای مثال، از حالتی که در شکل ۴-۱۲-الف نشان داده شده است، تنها با ۵ گام می توان به حالت شکل ۴-۱۲-ب رسید که دارای $h=1$ است و بسیار نزدیک به یک راه حل است. متأسفانه تپهنوردی اغلب به دلایل زیر گیر می کند:

- **پیشینه های محلی:** یک پیشینه محلی، قله ای است که از تمامی حالت های همسایه اش بلندتر ولی از پیشینه سراسری کوتاه تر است. الگوریتمهای تپهنوردی که به همسایگی یک پیشینه محلی می رسند، رو به بالا به سمت قله کشیده می شوند، ولی پس از آن گیر می کنند و جای دیگری نمی روند. شکل ۴-۱۰ این مسأله را به صورت شماتیک نمایش می دهد. حالت شکل ۴-۱۲-ب در حقیقت، یک پیشینه محلی است (یعنی یک کمینه محلی برای هزینه h). حرکت هر وزیر وضعیت را بدتر می کند.



(ب)



(الف)

شکل ۴-۱۲ (الف) یک حالت مسأله ۸ وزیر با برآورد هزینه آروین $h=17$ را نشان می دهد. برای هر پسین ممکن، مقدار h از حرکت وزیر در ستون خودش به دست می آید. در اینجا بهترین حرکتهای مشخص شده اند. (ب) یک کمینه محلی در فضای حالت ۸ وزیر. این حالت دارای $h=1$ است اما پسینها دارای هزینه بالاتری هستند.

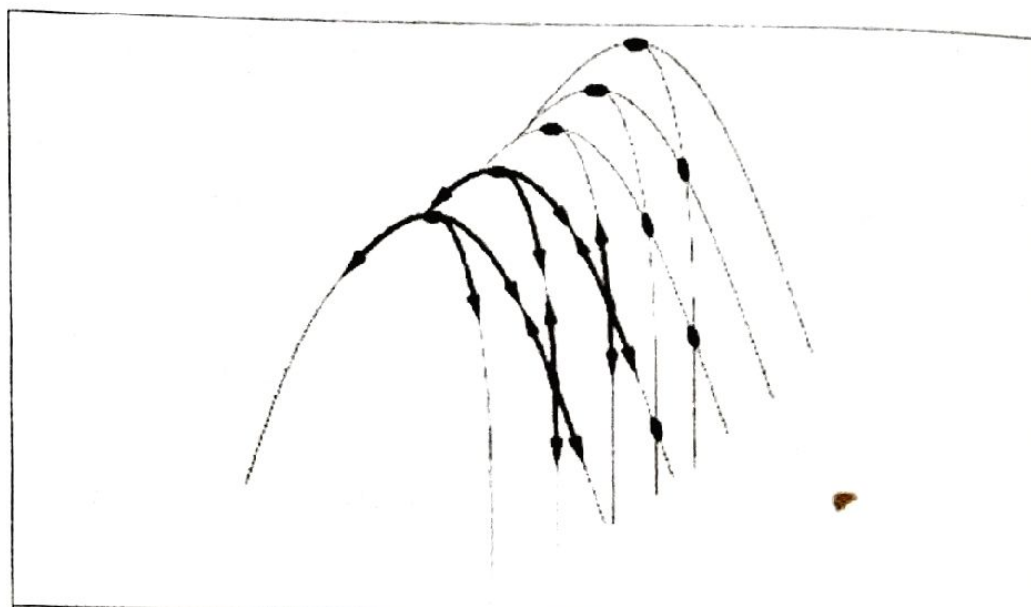
ف $\Rightarrow$ E	
Randomly	به صورت تصادفی
Greedy local search	جستجوی حریصانه محلی
Greedy	حرص
Seven deadly sins	هفت گناه کبیره
Local Maxima	پیشینه های محلی
Peak	قله
Situation	وضعیت
Successor	پسین
Local minimum	کمینه محلی
Ridge	دماغه
Plat	فلات
Evaluation function	تابع ارزیاب
Shoulder	شانه
Flat	مسطح
Hill-climbing	تپهنوردی
Landscape	دورنما

E ⇔ F	پیشرفت
Progress	پیشرفت
Instance	نمونه
Ridge	دماغه
Sideways move	حرکت از کناره
Shoulder	شانه
Positive	مثبت
Infinite	بی‌نهایت
Get stuck	گیر کردن
Limit	محدودیت
Allowed number	تعداد مجاز
To invent	ابداع کردن
Stochastic hill climbing	تپه‌نوردی اتفاقی
Flat local maximum	بیشینه مسطح
Successful	موفق
8-queens problem	مسأله ۸ وزیر
Percentage	درصد
Solved problems	مسائل حل شده

• **دماغه‌ها:** شکل ۴-۱۳ یک دماغه را نشان می‌دهد. دماغه‌ها دارای یک رشته بیشینه محلی هستند که گذشتن از آنها برای الگوریتمهای حریصانه بسیار مشکل است.

• **فلاتها:** فلات ناحیه‌ای از دورنمای فضای حالت است که در آن تابع ارزیاب، ثابت است. فلات می‌تواند یک بیشینه محلی مسطح باشد که از آن هیچ مسیر رو به بالایی وجود ندارد یا یک شانه باشد که از آن بتوان به بالاتر هم رفت. (به شکل ۴-۱۰ نگاه کنید.) یک جستجوی تپه‌نوردی ممکن است قادر نباشد که راه خروج از فلات را بیابد.

در هر کدام از این حالتها، الگوریتم به نقطه‌ای می‌رسد که دیگر پیشرفتی حاصل نمی‌شود. با شروع از یک حالت تصادفی ۸ وزیر، تپه‌نوردی به روش تیزترین صعود، در ۸۶ درصد اوقات گیر می‌کند و تنها ۱۴ درصد از مسائل را حل می‌کند. این روش خیلی سریع عمل می‌کند. در موارد موفق، به طور متوسط فقط ۴ گام و در مواردی که گیر می‌کند فقط ۳ گام برمی‌دارد (برای فضای حالتی که 8^8 یا حدود ۱۷ میلیون حالت دارد بد نیست).



شکل ۴-۱۳ نمایش این مطلب که چرا دماغه‌ها، برای تپه‌نوردی مشکل ایجاد می‌کنند. شبکه حالتها (دایره‌های مشکی) روی یک دماغه قرار داده شده‌اند با صعود از چپ به راست، یک رشته بیشینه محلی ایجاد می‌کنند که به طور مستقیم به یکدیگر متصل نیستند. تمامی اقدامات موجود در هر بیشینه محلی، رو به پایین هستند.

الگوریتم شکل ۴-۱۱ اگر به یک فلات برسد، قفل می‌کند. در آنجا بهترین پسین، مقداری همانند حالت فعلی دارد. آیا فکر خوبی نیست اگر مانند شکل ۴-۱۰ به راهمان ادامه دهیم (حرکت از کناره به امید این که این فلات در حقیقت یک شانه است)؟ جواب معمولاً مثبت است ولی باید مواظب باشیم. اگر همیشه در نبود حرکت رو به بالا، به حرکت‌های از کناره پردازیم، در صورتی که الگوریتم به یک بیشینه مسطح محلی غیر شانه برسد، در یک حلقه برونهایت گیر می‌کند. یک راه حل معمول، محدود کردن تعداد مجاز حرکت‌های از کناره پشت سرهم می‌باشد. برای مثال در مسأله ۸ وزیر می‌توان تا ۱۰۰ حرکت از کناره ادامه داد. این کار

درصد مسائل حل شده را از ۱۹٪ به ۹۴٪ می‌رساند. موفقیت همیشه دارای هزینه است؛ این الگوریتم در هر مورد موفق به طور متوسط به ۲۱ گام و در هر شکست به ۶۴ گام نیاز دارد.

تاکنون انواع متعددی از جستجوی تپهنوردی ابداع گردیده است. تپهنوردی اتفاقی از میان حرکت‌های رو به بالا به صورت تصادفی یکی را انتخاب می‌کند. احتمال این انتخاب می‌تواند براساس شیب حرکت‌های رو به بالا تغییر کند. این روش معمولاً کندتر از تیزترین شیب به نتیجه می‌رسد. اما در بعضی از دورنماهای حالت، بهترین راه‌حل را پیدا می‌کند. تپهنوردی اولین گزینه، تپهنوردی اتفاقی را به کار می‌گیرد به این صورت که به صورت تصادفی پسین تولید می‌کند تا زمانی که پسینی تولید شود که از حالت فعلی بهتر باشد. این راهبرد هنگامی مناسب است که یک حالت دارای تعداد زیادی (مثلاً هزار) پسین باشد. در تمرین ۴-۱۶ می‌توانید در این باره بیشتر مطالعه کنید.

الگوریتم‌های تپهنوردی که تاکنون توضیح داده شد همگی ناکامل هستند (اغلب موفق نمی‌شوند اهداف موجود را بیابند زیرا ممکن است در یک بیشینه محلی گیر کنند). تپهنوردی با شروع مجدد تصادفی با بهره‌گیری از ضرب‌المثل "در ناامیدی بسی امید است"، یک مجموعه جستجوی تپهنوردی را با حالت‌های شروع تصادفی اجرا می‌کند^۸ و هنگامی که یک هدف پیدا شد متوقف می‌شود. این روش با احتمال نزدیک به یک، کامل می‌باشد و برای این موضوع دلیل ساده‌ای وجود دارد: بالاخره یک حالت هدف به عنوان حالت شروع تولید خواهد شد. اگر هر جستجوی تپهنوردی احتمال موفقیت p داشته باشد، مقدار متوسط تعداد شروع‌های مجدد لازم $\frac{1}{p}$ است. در مورد نمونه‌هایی مانند ۸ وزیر که هیچ حرکت از کناره‌ای مجاز نباشد، $p \approx 0.14$. بنابراین، حدود ۷ تکرار (۶ شکست و ۱ موفقیت) نیاز داریم تا یک هدف را بیابیم. مقدار متوسط تعداد گام‌های برابر است با هزینه یک تکرار موفق به علاوه $\frac{1-p}{p}$ بار هزینه شکست که تقریباً ۲۲ گام می‌شود. هنگامی که حرکت‌های از کناره مجاز باشد به طور متوسط به $1.06 \approx \frac{1}{0.94}$ بار تکرار و $25 \approx (1 \times 21) + (0.06 / 0.94) \times 64$ گام نیاز است. در نتیجه، در مورد ۸ وزیر، تپهنوردی با شروع مجدد تصادفی واقعاً مؤثر است. حتی برای سه میلیون وزیر هم این روش می‌تواند در کمتر از یک دقیقه راه‌حل پیدا کند^۹.

موفقیت تپهنوردی، بستگی زیادی به شکل دورنمای فضای حالت دارد: اگر تعداد بیشینه محلی و فلات کم باشد، تپهنوردی با شروع مجدد تصادفی یک راه‌حل خوب را خیلی سریع پیدا می‌کند. از طرف دیگر، بسیاری از مسائل واقعی دارای دورنمایی هستند که بیشتر شبیه یک خانواده جوجه‌تیغی است که روی یک سطح صاف قرار گرفته‌اند. همراه با جوجه‌تیغی‌های مینیاتوری که بر روی نوک هر تیغ این جوجه‌تیغی‌ها زندگی می‌کنند و الی آخر. مسائل NP-سخت معمولاً دارای تعداد نمایی بیشینه محلی هستند که در آنها گیر می‌افتند. با وجود این،

$E \Rightarrow$	ف
Steepness	شیب
Slow	کند
First-choice hill climbing	تپهنوردی اولین گزینه
Implicit	ضمنی
Incomplete	ناکامل
Random-restart hill climbing	تپهنوردی با شروع مجدد تصادفی
Adage	ضرب‌المثل
To stop	متوقف کردن
Exponential number	تعداد نمایی
Restart	شروع مجدد
Expected number of steps	مقدار متوسط تعداد گام‌ها
Effective	مؤثر
Local maximum	بیشینه محلی
Plateau	فلات

^۸ تولید یک حالت تصادفی از یک فضای حالت که به صورت ضمنی تعریف شده است، می‌تواند به خودی خود یک مسأله سخت باشد.

^۹ Luby et al. (1993) ثابت کردند که در بعضی موارد، بهترین کار این است که پس از مدت زمان ثابتی، یک الگوریتم جستجوی تصادفی را مجدداً شروع کنیم و این عمل می‌تواند بسیار مؤثرتر از این باشد که اجازه دهیم هر جستجو به صورت نامحدود به کارش ادامه دهد. جلوگیری یا محدودیت تعداد حرکت‌های از کناره یک مثال از این مطلب می‌باشد.

یک پیشینه محلی نسبتاً خوب را اغلب می‌توان پس از تعداد کمی شروع مجدد پیدا کرد.

جستجوی شبه تاب کاری

از آنجا که الگوریتم تپه‌نوردی هرگز حرکت "رو به پایین" به سمت حالت‌هایی با مقادیر کمتر (یا هزینه بالاتر) را انجام نمی‌دهد، ناکامل بودنش قطعی است، زیرا ممکن است در یک پیشینه محلی گیر کند. در مقابل، یک راه‌پیمایی تصادفی محض (یعنی حرکت به پسینی از مجموعه پسینها که به طور یکنواخت و کاملاً تصادفی انتخاب شده است) کامل ولی به شدت ناکارآمد می‌باشد. در نتیجه، به منظور کارآمدی و کامل شدن، ترکیب تپه‌نوردی با یک راه‌پیمایی تصادفی در قالب شبه تاب کاری تلاش معقولی به نظر می‌رسد. در فلزشناسی برای آب دادن یا بادوام کردن، با استفاده از فرآیند سخت‌سازی، فلزات و شیشه‌ها را تا دمای بسیار بالا داغ می‌کنند و سپس تدریجاً خنک می‌کنند که در نتیجه، به ماده اجازه می‌دهد که به یک حالت کریستالی با انرژی کم تبدیل شود. برای درک شبه تاب کاری اجازه بدهید به جای تپه‌نوردی، به پایین آمدن یک توپ بینگ‌پنگ از شیب (یعنی کمینه کردن هزینه) به درون عمیق‌ترین شکاف در یک سطح ناهموار توجه کنیم. اگر فقط توپ را رها کنیم تا قبل برخورد، در یک کمینه محلی متوقف خواهد شد. اگر سطح را تکان دهیم می‌توانیم توپ را از کمینه محلی بیرون آوریم. نکته در این جااست که فقط به اندازه‌ای محکم تکان دهیم که توپ را از کمینه محلی بیرون اندازد، نه آن قدر محکم که آن را از یک کمینه سراسری نیز درآورد. راه حل شبه تاب کاری با تکانهای شدید (مشابه حرارت بسیار بالا) آغاز می‌شود و سپس، تدریجاً شدت تکانها کاهش می‌یابد (مشابه دمای کمتر).

ف	E
Incomplete	ناکامل
Pure random walk	راه‌پیمایی تصادفی محض
Uniform	یکنواخت
inefficient	ناکارآمد
Simulated annealing	شبه تاب کاری
Metallurgy	فلزشناسی
To temper	آب دادن
To harden	بادوام نمودن
Metals	فلزات
Glass	شیشه
Gradient descent	پایین آمدن از شیب
Crevise	شکاف
Bumpy surface	سطح ناهموار
Shaking	تکان
Allowed	مجاز
Similar	شبه
Badness	میزان بدی
Probability	احتمال
Intense	شدید

```

function SIMULATED-ANNEALING(problem, schedule) returns a solution state
inputs: problem, a problem
       schedule, a mapping from time to "temperature"
local variables: current, a node
                 next, a node
                 T, a "temperature" controlling the probability of downward steps

current ← MAKE-NODE(INITIAL-STATE[problem])
for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
    ΔE ← VALUE[next] - VALUE[current]
    if ΔE > 0 then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 

```

شکل ۴-۱۴ الگوریتم جستجوی شبه تاب کاری، نسخه‌ای از تپه‌نوردی اتفاقی است که در آن بعضی حرکت‌های رو به پایین مجاز می‌باشند. حرکت‌های رو به پایین در اوایل زمان‌بندی تاب کاری به سادگی پذیرفته می‌شوند و هر چه زمان می‌گذرد، کمتر قابل پذیرش هستند و ورودی $schedule$ مقدار T را به صورت تابعی از زمان تعیین می‌کند.

داخلی‌ترین حلقه الگوریتم شبه تاب‌کاری (شکل ۴-۱۴) کاملاً شبیه تپه‌نوردی است. با این تفاوت که به جای انتخاب بهترین حرکت، یک حرکت تصادفی را انتخاب می‌کند. اگر این حرکت، شرایط را بهبود ببخشد، همیشه پذیرفته می‌شود. در غیر این صورت، الگوریتم این حرکت را با احتمالی کمتر از ۱ می‌پذیرد. این احتمال با "میزان بدی" حرکت (ΔE) که براساس آن وضعیت بدتر می‌شود، به صورت نمایی کاهش می‌یابد. این احتمال همچنین با کاهش دمای T کاهش می‌یابد: احتمال مجاز شمردن حرکت‌های بد در آغاز که دما بالاست، زیاد است و با کاهش T ، آن نیز کاهش می‌یابد. می‌توان ثابت کرد که اگر زمان‌بندی T را به اندازه کافی آهسته کاهش دهد، این الگوریتم یک بهینه سراسری را با احتمال نزدیک به ۱ پیدا خواهد کرد.

شبه تاب‌کاری اولین بار در اوایل دهه ۱۹۸۰ برای حل مسائل چیدمان $VLSI$ به طور گسترده‌ای مورد استفاده قرار گرفت. این الگوریتم در زمان‌بندی کارخانه‌ها و دیگر کارهای بهینه‌سازی در مقیاس بزرگ به کار می‌رود. در تمرین ۴-۱۶ از شما خواسته شده است که در مسأله n وزیر، کارایی آن را با کارایی تپه‌نوردی با شروع مجدد تصادفی مقایسه کنید.

جستجوی پرتوی محلی

ممکن است نگهداری تنها یک گره در حافظه یک واکنش افراطی نسبت به مسأله محدودیت حافظه به نظر می‌رسد. در الگوریتم جستجوی پرتوی محلی^۱ به جای یک حالت، اطلاعات k حالت نگهداری می‌شود. این الگوریتم با k حالت تصادفی شروع به کار می‌کند. در هر گام، تمامی پسینهای هر k حالت تولید می‌شوند. هر کدام از آنها که هدف باشد، الگوریتم متوقف می‌شود. در غیر این صورت، با انتخاب بهترین k پسین از کل لیست، این عملیات را تکرار می‌کند.

در نگاه اول، جستجوی محلی با k حالت ممکن است چیزی بیش از اجرای موازی k شروع مجدد تصادفی به جای پشت سر هم به نظر نرسد. ولی در حقیقت، این دو الگوریتم کاملاً با یکدیگر تفاوت دارند. در یک جستجو با شروع مجدد تصادفی، هر فرآیند جستجو به طور مستقل از بقیه اجرا می‌شود. در یک جستجوی پرتوی محلی، اطلاعات سودمند در بین k رشته جستجوی موازی رد و بدل می‌گردد. برای مثال، اگر یک حالت، چندین پسین خوب تولید کند و $k-1$ حالت دیگر همگی پسینهای بدی تولید کنند، نتیجه این می‌شود که حالت اول به بقیه اعلام می‌کند: اینجا بیاید، بهترینها اینجا هستند! الگوریتم به سرعت جستجوهای بدون ثمر را ترک می‌کند و منابع خود را به جایی منتقل می‌کند که بیشترین پیشرفت صورت می‌گیرد.

جستجوی پرتوی محلی به همان شکل اولیه خود ممکن است از نبود تنوع در بین k حالت رنج ببرد (یعنی امکان دارد به سرعت تمامی آنها در یک منطقه کوچک از فضای حالت متمرکز شوند و جستجو را به یک نوع تپه‌نوردی پرهزینه تبدیل کنند). یک ویرایش دیگر از این الگوریتمها که جستجوی پرتوی اتفاقی نامیده می‌شود و قابل قیاس با تپه‌نوردی اتفاقی است، به رفع این مشکل کمک می‌کند. جستجوی پرتوی اتفاقی به جای انتخاب k بهترین از مجموعه پسینهای منتخب، k پسین را به صورت تصادفی انتخاب می‌کند، به طوری که احتمال انتخاب یک

$E \rightarrow$	ف
Evaluation	ارزیابی
Exponentially	به صورت نمایی
Temperature	دما
Schedule	زمان‌بندی
Layout	چیدمان
Performance	کارایی
Random-restart hill climbing	تپه‌نوردی با شروع مجدد تصادفی
Node	گره
Local beam search	جستجوی پرتوی محلی
Stochastic beam search	جستجوی پرتوی اتفاقی
Parallel	موازی

^۱ جستجوی پرتوی محلی، اقتباسی است از جستجوی پرتوی که یک الگوریتم مبتنی بر مسیر است.

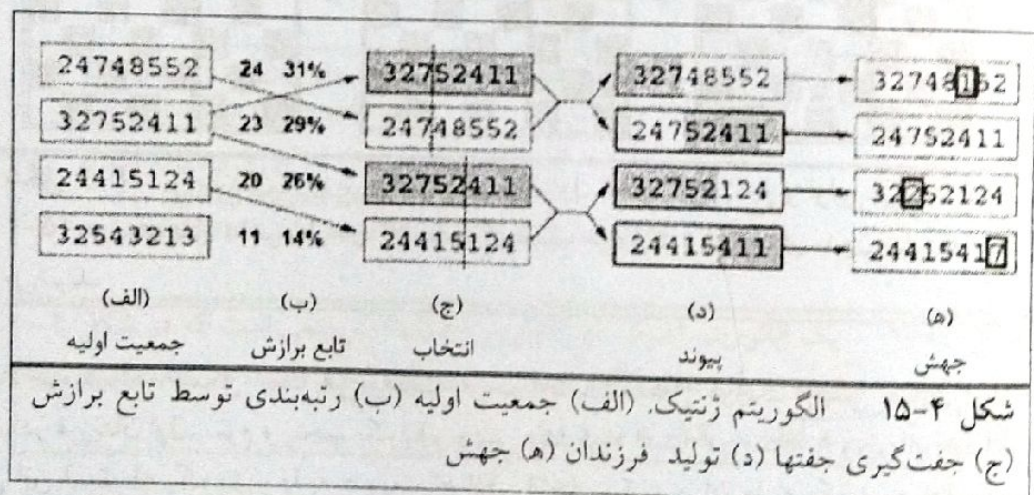
$E \Rightarrow$	ف
Genetic algorithm	الگوریتم ژنتیک
Asexual reproduction	تولید مثل غیر جنسی
Population	جمعیت
Individual	فرد
Alphabet	الفبا
Column	ستون
Square	مربع
Digit	رقم
Encoding	رمزگذاری
Evaluation function	تابع ارزیاب
Fitness function	تابع برازش
Rating	رتبه‌بندی
To attack	حمله کردن
Particular variant	نمونه خاص
Raw scores	نمرات خام
Pair	جفت
Solution	راه‌حل

پسین خاص به صورت یک تابع افزایشی باشد. جستجوی پرتوی اتفاقی شباهتهایی به فرآیند انتخاب طبیعی دارد که در آن پسینهای (فرزندان) یک حالت (موجود زنده) نسل بعدی را متناظر با مقدارش (برازش آن) تشکیل می‌دهد.

الگوریتمهای ژنتیک

الگوریتم ژنتیک (GA) یک گونه از جستجوی پرتوی اتفاقی است که حالت‌های پسین به جای تغییر فقط یک حالت، از ترکیب دو حالت والد به وجود می‌آید. همانند جستجوی پرتوی اتفاقی، این روش نیز قابل مقایسه با انتخاب طبیعی است، با این تفاوت که با تولید مثل جنسی سرو کار داریم نه غیر جنسی.

همانند جستجوی پرتوی، GA با یک مجموعه از k حالت تصادفی که جمعیت نامیده می‌شود شروع به کار می‌کند. هر حالت، یا فرد، به صورت یک رشته از الفبای محدود (معمولاً رشته‌ای از ۰ و ۱) بیان می‌شود. برای مثال، یک حالت ۸ وزیر باید موقعیت هر یک از ۸ وزیر را در یک مربع ۸ ستونی تعیین کند و بنابراین به $24 = 8 \times \log_2 8$ بیت نیاز داریم. راه دیگر این است که یک حالت را با ۸ رقم مختلف که هر یک در محدوده ۱ تا ۸ قرار می‌گیرند، بیان کرد. (بعداً مشاهده خواهیم کرد که این دو روش رمزگذاری رفتارهای مختلفی دارند.) شکل ۴-۱۵ الف یک جمعیت از چهار رشته ۸ رقمی را نشان می‌دهد که هر یک بیان‌گر یک حالت ۸ وزیر هستند.

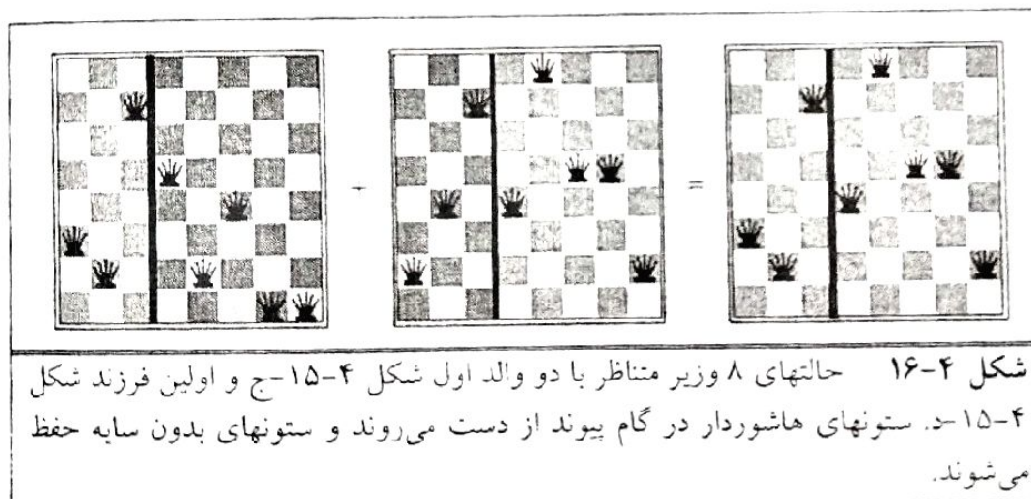


تولید نسل بعدی حالتها در شکل ۴-۱۵ ب تا ۴-۱۵ ه نشان داده شده است. در (ب) هر از حالتها توسط تابع ارزیاب یا (به زبان الگوریتمهای ژنتیک) تابع برازش رتبه‌بندی می‌شود. یک تابع برازش باید برای حالتها بهتر، مقادیر بزرگ‌تر برگرداند. بنابراین در مورد مسأله ۸ وزیر از تعداد جفت وزیرهایی که به هم حمله نمی‌کنند استفاده می‌کنیم که برای یک راه‌حل مقدار ۲۸ خواهد بود. مقادیر چهار حالت، ۲۴ و ۲۳ و ۲۰ و ۱۱ است. در این گونه خاص از الگوریتم ژنتیک، احتمال انتخاب برای تولید مثل مستقیماً متناسب است با نمره برازش و درصدها در کنار نمرات خام نمایش داده شده‌اند.

در (ج) دو جفت به صورت تصادفی، مطابق با احتمالات (ب) برای تولید مثل انتخاب

شده است. توجه داشته باشید که یک مورد دوبار انتخاب شده است و مورد دیگری اصلاً انتخاب نشده است.^{۱۱} برای هر جفتی که باید جفت‌گیری کنند، یک نقطه پیوند به صورت تصادفی از موقعیتهای داخل رشته انتخاب می‌شود. در شکل ۴-۱۵ نقاط پیوند جفت اول پس از سومین رقم و در جفت دوم پس از پنجمین رقم قرار می‌گیرند.^{۱۲}

در (د) فرزندان از برخورد رشته‌های والد در نقطه پیوند به وجود می‌آیند. برای مثال، فرزند اول جفت اول، سه رقم اولش را از والد اول و بقیه رقمهایش را از والد دوم می‌گیرد، در حالی که فرزند دوم سه رقم اولش را از والد دوم و بقیه رقمهایش را از والد اول دریافت می‌کند. در این مرحله، حالت‌های مسأله ۸ وزیر در شکل ۴-۱۶ نشان داده شده است. این مثال، نشان‌دهنده این واقعیت است که هنگامی که دو حالت والد کاملاً متفاوت هستند، عمل پیوند ممکن است حالتی را تولید کند که با هر دو والدش تفاوت بسیار زیادی داشته باشد. اغلب در اوایل این فرآیند جمعیت کاملاً متنوع است. بنابراین، پیوند (همانند شبه تاب‌کاری) غالباً در اوایل فرآیند جستجو، گامهای بزرگی در فضای حالت برمی‌دارد و بعدها زمانی که اکثر نمونه‌ها کاملاً مشابه هم می‌شوند، گامهای کوچک‌تری برمی‌دارد.



و بالاخره در (ه)، ممکن است هر موقعیت با یک احتمال مستقل کوچک جهش تصادفی پیدا کند. در فرزندان اول، سوم و پنجم، یک رقم جهش پیدا کرده است. در مسأله ۸ وزیر، این مسأله مثل این است که یک وزیر را به صورت تصادفی انتخاب کنیم و آن را به یک مربع خالی در همان ستون منتقل کنیم. شکل ۴-۱۷ الگوریتمی را نشان می‌دهد که تمامی این گامها را پیاده‌سازی می‌کند.

الگوریتمهای ژنتیک همانند جستجوی پرنوی اتفاقی، میل به بالا رفتن را با کاوش تصادفی و تبادل اطلاعات میان رشته‌های موازی جستجو ترکیب می‌کنند. اگر الگوریتمهای ژنتیک مزیتی

^{۱۱} قوانین انتخاب بسیار متنوعی وجود دارد. می‌توان نشان داد روش وجین که در آن تمامی مواردی که پایین‌تر از یک آستانه خاص قرار می‌گیرند دور انداخته می‌شوند، سریع‌تر از نسخه تصادفی هم‌گرا می‌شود (Baum et al., 1995).

^{۱۲} اینجاست که نحوه رمزگذاری اهمیت پیدا می‌کند. اگر به جای ۸ رقم، از رمزگذاری ۲۴ بیتی استفاده شود، در آن صورت نقطه پیوند به احتمال ۲/۳ در وسط یک رقم قرار می‌گیرد که به جهش اختیاری آن رقم منتهی می‌شود.

E	ف
Reproduction	تولید مثل
To mate	جفت‌گیری
Crossover point	کردن
Digit	نقطه پیوند
Different	رقم
Population	متفاوت
Parent	جمعیت
Shaded columns	والد
Mutation	ستونهای هاشوردار
Culling method	جهش
To converge	روش وجین
Random version	همگرا شدن
Arbitrary	نسخه تصادفی
To implement	اختیاری
	بیاده‌سازی کردن

E	ف
Mating	جفت‌گیری
Parent	والد
Uphill tendency	میل به بالا رفتن
Random exploration	کاوش تصادفی
Advantage	مزیت
Level of granularity	درجه دانهای بودن
A useful block	یک قطعه مفید
Schema	طرح
Substring	زیررشته
Instance	نمونه
Fitness	برازش
Population	جمعیت
Consistency	سازگاری
Meaningful	معنادار
To correspond	مطابقت داشتن
Antenna	آنتن
Reflector	بازتابنده
deflector	منحرف‌کننده

داشته باشند. مزیت اصلی آنها مربوط به عمل پیوند است. با وجود این، می‌توان از نظر ریاضی نشان داد که اگر موقعیت رمز ژنتیک در ابتدا به طور تصادفی تغییر داده شود، پیوند هیچ مزیتی ندارد. مزیت پیوند، ناشی از توانایی پیوند در ترکیب قطعه‌های بزرگی از حروف می‌باشد که به طور مستقل تکامل پیدا کرده‌اند تا رفتار مفیدی داشته باشد و در نتیجه، درجه دانهای بودن که جستجو در آن عمل می‌کند را افزایش می‌دهند. برای مثال، قرار دادن سه وزیر اول در موقعیت‌های ۲ و ۴ و ۶ (که به هم حمله نمی‌کنند) یک قطعه مفید است که می‌تواند با قطعه‌های دیگر ترکیب شود تا یک راه‌حل را بسازد.

function GENETIC-ALGORITHM(*population*, FITNESS-FN) returns an individual
 inputs: *population*, a set of individuals
 FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population ← empty set

loop for *i* **from** 1 **to** SIZE(*population*) **do**

x ← RANDOM-SELECTION(*population*, FITNESS-FN)

y ← RANDOM-SELECTION(*population*, FITNESS-FN)

child ← REPRODUCE(*x*, *y*)

if (small random probability) **then** *child* ← MUTATE(*child*)

add *child* **to** *new_population*

population ← *new_population*

until some individual is fit enough, or enough time has elapsed.

return the best individual in *population*, according to FITNESS-FN

function REPRODUCE(*x*, *y*) returns an individual

inputs: *x*, *y*, parent individuals

n ← LENGTH(*x*)

c ← random number from 1 to *n*

return APPEND(SUBSTRING(*x*, 1, *c*), SUBSTRING(*y*, *c* + 1, *n*))

شکل ۴-۱۷ یک الگوریتم ژنتیک. این الگوریتم همان الگوریتمی است که در شکل ۴-۱۵ به تصویر درآمده با یک تفاوت که: در این نسخه که رایج‌تر است، هر جفت‌گیری دو والد تنها یک فرزند تولید می‌کند، نه دو فرزند.

نظریه الگوریتم‌های ژنتیک با استفاده از ایده طرح که زیررشته‌ای است که در آن چند موقعیت می‌توانند تعیین نشده باقی بمانند، این مسأله را توضیح می‌دهد. برای مثال، طرح ۲۴۶***** برای تمامی حالت‌های ۸ وزیر را که در آنها اولین سه وزیر به ترتیب در موقعیت‌های ۲ و ۴ و ۶ قرار دارند توصیف می‌کند. رشته‌هایی که با این طرح مطابقت دارند (مانند ۲۴۶۱۲۵۷۸) نمونه‌های این طرح نامیده می‌شوند. می‌توان نشان داد که اگر متوسط برازش نمونه‌های یک طرح بالاتر از میانگین باشد، تعداد نمونه‌های طرح در بین جمعیت، در طول زمان افزایش خواهد یافت. واضح است که اگر بیت‌های مجاور کاملاً با یکدیگر مرتبط باشند، تأثیر فوق احتمالاً ناچیز خواهد بود، زیرا در آن صورت تعداد کمی قطعات چسبیده به هم وجود خواهد داشت که دارای مزیت سازگاری باشد. الگوریتم‌های ژنتیک زمانی بهتر از همیشه عمل می‌کنند که طرح‌ها با اجزای

معناداری از راه حل مطابقت داشته باشند. برای مثال، اگر رشته بیان گر یک آنتن باشد، این طرحها می توانند قطعات آنتن، از قبیل بازتابنده و منحرف کننده باشند. یک جزء خوب به احتمال زیاد در انواع مختلف طراحیها مناسب است. این بدان معنی است که برای استفاده موفقیت آمیز از الگوریتمهای ژنتیک، نیازمند مهندسی دقیق بازنمایی است.

در عمل، الگوریتمهای ژنتیک در مسائل بهینه سازی از قبیل چیدمان مدار و برنامه ریزی زمانی کارگاه بسیار پرکاربرد هستند. در حال حاضر، مشخص نیست که جذابیت الگوریتمهای ژنتیک از عملکرد آنها ناشی می شود یا از خاستگاه زیبا و دلپذیر آنها که نظریه تکامل می باشد. کارهای زیادی هنوز باقی مانده است که برای شناسایی شرایطی که تحت آنها الگوریتمهای ژنتیک خوب عمل می کنند، در حال انجام می باشد.

۴-۴ جستجوی محلی در فضاها پیوسته

در فصل ۲، تفاوت بین محیطهای گسسته و پیوسته را توضیح دادیم و تأکید کردیم که اکثر محیطهای دنیای حقیقی پیوسته هستند. با این حال، هیچ کدام از الگوریتمهایی که تاکنون بیان کرده ایم نمی توانند از پس فضاها پیوسته برآیند. (در اکثر موارد، تابع پسین تعداد نامتناهی حالت برمی گرداند) این بخش مقدمه بسیار مختصری درباره بعضی فنون جستجوی محلی برای پیدا کردن راه حلهای بهینه در فضاها پیوسته ارائه می کند. ادبیات این موضوع بسیار گسترده است. بسیاری از فنون اولیه در قرن هفدهم، پس از به وجود آمدن حسابان توسط نیوتن و لایبنیتز ایجاد شدند^{۱۳}. در بخشهای مختلف این کتاب، یعنی فصلهای مربوط به یادگیری، بینایی و رباتیک کاربردهای این فنون را خواهیم یافت.

اجازه دهید با یک مثال آغاز کنیم. فرض کنید می خواهیم سه فرودگاه جدید در رومانی بسازیم، به نحوی که مجموع مجذور فواصل تمام شهرهای روی نقشه (شکل ۳-۲) از نزدیکترین فرودگاه به آن کمینه شود. در این صورت، فضای حالت توسط مختصات این فرودگاهها بیان می شود: (x_1, y_1) و (x_2, y_2) و (x_3, y_3) . این یک فضای شش بعدی است. همچنین، می گوئیم که حالتها توسط ۶ متغیر بیان می شوند (در حالت کلی، حالتها توسط یک بردار n بعدی از متغیرهای x بیان می شوند) جابه جا شدن در این فضا، متناظر است با جابه جایی یک یا چند فرودگاه بر روی نقشه. محاسبه تابع هدف $f(x_1, y_1, x_2, y_2, x_3, y_3)$ برای هر حالت خاص، اگر قبلاً نزدیکترین شهرها را محاسبه کرده باشیم، نسبتاً ساده است. ولی اگر بخواهیم در حالت کلی بنویسیم، نسبتاً پیچیده می شود.

یک روش برای اجتناب از مسائل پیوسته، گسسته سازی همسایگی هر حالت می باشد. برای مثال، می توانیم تنها یک فرودگاه را در هر زمان در یکی از جهات x یا y به اندازه مقدار ثابت $\pm \delta$ جابه جا کنیم. بدین ترتیب برای ۶ متغیر، به ازای هر حالت ۱۲ پسین خواهیم داشت. سپس، می توان یکی از الگوریتمهای جستجوی محلی را که پیش از این بیان کردیم، به کاربرد. همچنین، می توان نپهنوردی و شبه تاب کاری را مستقیماً بدون گسسته سازی فضا به کاربرد. این الگوریتمها، پسینها را به طور تصادفی انتخاب می کنند که این کار می تواند با تولید تصادفی

 ف $\Rightarrow$ E

Successful

موفقیت آمیز

Coordinates

مختصات

Variable

متغیر

Vector

بردار

Circuit layout

چیدمان مدار

Job-shop scheduling

برنامه ریزی زمانی

کارگاه

Theory of evolution

نظریه تکامل

Discrete

گسسته

Continuous

پیوسته

Infinite

نامتناهی

Local search techniques

فنون جستجوی

محلی

Newton

نیوتن

Leibniz

لایبنیتز

Learning

یادگیری

Vision

بینایی

Sum of squared distances

مجموع مجذور

فواصل

^{۱۳} هنگام مطالعه این بخش، شناخت اولیه از حسابان چندمتغیره و حساب برداری مفید می باشد.

بردارهایی به طول δ انجام پذیرد.

روشهای زیادی وجود دارد که از گرادیان دورنما برای یافتن یک بیشینه استفاده می کنند. گرادیان تابع هدف بردار ∇f می باشد که بزرگی و جهت تیزترین شیب را می دهد. برای این مسأله داریم:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

در بعضی موارد، می توانیم از طریق حل معادله $\nabla f = 0$ یک بیشینه پیدا کنیم (مثلاً اگر بخواهیم فقط یک فرودگاه بسازیم جواب، میانگین حسابی مختصات تمامی شهرهاست). در بسیاری از موارد، این معادله به شکل بسته قابل حل نیست. برای مثال، با سه فرودگاه، این عبارت برای گرادیان به این بستگی دارد که در حالت فعلی چه شهرهایی به هر فرودگاهی از همه نزدیک تر می باشند. یعنی، می توانیم گرادیان را به صورت محلی محاسبه کنیم، نه سراسری. حتی در این صورت نیز می توان الگوریتم تپه نوردی را با استفاده از تیزترین شیب با به روزآوری حالت فعلی از طریق فرمول زیر انجام داد. (α یک ثابت کوچک است):

$$x \leftarrow x + \alpha \nabla f(x)$$

در موارد دیگر، ممکن است یک تابع هدف مشتق پذیر اصلاً موجود نباشد (برای مثال، ارزش اقتصادی یک مجموعه خاص از موقعیتهای فرودگاهها، می تواند با شبیه سازی اقتصادی توسط بسته های نرم افزاری در مقیاس وسیع تعیین شود). در این موارد، به اصطلاح یک گرادیان تجربی با ارزیابی پاسخ نسبت به افزایش و کاهشهای کوچک در هر یک از مختصات تعیین می شود. در یک نسخه گسسته شده از فضای حالت، جستجوی گرادیان تجربی مشابه تپه نوردی با روش تیزترین شیب است.

در بطن عبارت " α یک ثابت کوچک است"، روشهای مختلفی برای تنظیم α وجود دارد. مسأله اصلی این است که اگر α بیش از حد کوچک باشد، به تعداد بسیار زیادی گام نیاز خواهیم داشت و اگر α بیش از حد بزرگ باشد، جستجو ممکن است از بیشینه بالاتر برود. فن جستجوی خطی تلاش می کند که تا زمانی که گام شروع به کاهش مجدد کند، با گسترش جهت گرادیان فعلی (معمولاً از طریق دو برابر کردن مکرر α)، بر این تنگنا فائق آید. نقطه ای که در آن چنین اتفاقی رخ می دهد، جایگزین حالت فعلی می شود. مکاتب فکری متعددی درباره چگونگی انتخاب جهت جدید در این مرحله وجود دارد.

روش ارزشمند نیوتن-رافسون (Newton, 1671; Raphson, 1690) یکی از مؤثرترین الگوریتمها در بسیاری از مسائل می باشد. این روش یک فن عمومی برای یافتن ریشه های توابع (یعنی حل معادلاتی به شکل $g(x)=0$) است. این الگوریتم با محاسبه تخمین جدید از ریشه x بر طبق فرمول ذیل عمل می کند:

$$x \leftarrow x - g(x) / g'(x)$$

$E \Rightarrow$	ف
Gradient	گرادیان
Steepest slope	تیزترین شیب
Maximum	بیشینه
Arithmetic mean	میانگین حسابی
Coordinates	مختصات
Closed	بسته
Local	محلی
Global	سراسری
Differentiable	مشتق پذیر
Economic simulation	شبیه سازی اقتصادی
Empirical gradient	گرادیان تجربی
Line search	جستجوی خطی
Repeated	مکرر
School of thought	مکاتب فکری
Newton-Raphson	نیوتن-رافسون
Landscape	دورنما
Root	ریشه

تکامل و جستجو

نظریه تکامل در مقاله *On the Origin of Species by Means of Natural Selection* نوشته *Charles Darwin (1859)* متولد شد. ایده اصلی این نظریه بسیار ساده است: گوناگونیها (که با عنوان جهش شناخته می‌شوند) در فرآیند تولید مثل، رخ می‌دهند و تقریباً به نسبت تأثیرشان بر برازش تناسلی، در نسلهای بعدی حفظ می‌شوند.

نظریه داروین بدون هیچ دانشی درباره چگونگی ارث‌بری و تغییر ویژگیهای موجودات زنده شکل گرفت. قوانین احتمالی که بر این فرایندها حاکم هستند برای اولین بار توسط راهبی به نام *Gregor Mendel (1866)* با آزمایشهای باروری مصنوعی بر روی نخود شیرین شناسایی گردید. پس از مدت طولانی، *Watson and Crick (1953)* ساختار مولکول *DNA* و الفبای آن، *AGTC (adenine, guanine, thymine, cytosine)* را شناسایی کردند. در مدل استاندارد، گوناگونی هم از طریق جهش حرف در توالی حروف اتفاق می‌افتد و هم از طریق پیوند (که در آن، *DNA* یک فرزند از طریق ترکیب بخشهایی از *DNA* هر یک از والدین تولید می‌شود).

قبلاً این موضوع را با الگوریتمهای جستجوی محلی قیاس کرده‌ایم. تفاوت اساسی بین جستجوی پرتوی اتفاقی و تکامل، استفاده از تولید مثل جنسی است، که در آن پسینها از چند موجود زنده می‌آیند، نه یکی. سازوکار واقعی تکامل بسیار غنی‌تر از آنچه که اکثر الگوریتمهای ژنتیک اجازه می‌دهند می‌باشد. برای مثال، تکامل می‌تواند شامل برگشت، تکثیر و جابه‌جایی قطعات بزرگ *DNA* باشد. بعضی ویروسها *DNA* را از یک موجود زنده قرض می‌گیرند و آن را در یک موجود زنده دیگر درج می‌کنند. ژنهای جابه‌جایی وجود دارند که کاری جز هزاران بار کپی کردن خودشان در ژنوم ندارند. حتی ژنهایی وجود دارند که سلولهای جفتی که دارای آن ژن نیستند را مسموم می‌کنند تا شانس تکثیر خود را افزایش دهند. مهم‌تر از همه این واقعیت است که ژنها خودشان این سازوکارها را به رمز درمی‌آورند که به وسیله آنها ژنوم تولید مثل و به یک موجود زنده ترجمه می‌شود. در الگوریتمهای ژنتیک، این ساز و کارها برنامه جداگانه‌ای هستند که درون رشته‌های مورد استفاده، بازنمایی نمی‌شوند.

تولید کورکورانه حدود 10^{45} موجود زنده، بدون هیچ بهبودی در آروین جستجو معقول به نظر نمی‌رسد و به همین دلیل، تکامل داروینی در این مورد قابل قبول نیست. با وجود این، پنجاه سال پیش از داروین، یک طبیعی‌دان فرانسوی بزرگ دیگر، *Jean Lamarck (1809)* نظریه تکاملی را عرضه کرد که در آن خصلتهایی که از طریق سازگاری در طول زندگی یک موجود زنده به دست می‌آید نیز به فرزندانش منتقل می‌شد. چنین فرآیندی می‌توانست مؤثر باشد، ولی به نظر نمی‌رسید که در طبیعت رخ دهد. سالها بعد، *James Baldwin (1896)* یک فرضیه ظاهراً مشابه را ارائه کرد: رفتاری که در طول زندگی یک موجود زنده آموخته می‌شود می‌تواند سرعت تکامل را افزایش دهد. برخلاف فرضیه لامارک، فرضیه بالدوین کاملاً با تکامل داروینی سازگار است. زیرا بر فشارهای انتخابی تکیه می‌کند که بر افرادی که یک بهینه محلی در میان مجموعه رفتارهای ممکن که آرایش ژنشان اجازه می‌دهد وارد می‌شود. شبیه‌سازیهای کامپیوتری اخیر تأیید می‌کنند که "اثر بالدوین" واقعی است. به شرطی که تکامل معمولی بتواند موجودات زنده‌ای خلق کند که میزان کارایی درونی‌شان تا حدودی به برازش واقعی‌شان بستگی داشته باشد.

E ⇔ F

Evolution

تکامل

Variations

گوناگونیها

Mutation

جهش

Reproduction

تولید مثل

Fitness

برازش

Inheritance

ارث‌بری

Generation

نسل

Darwin

داروین

Traits of

organisms

ویژگیهای

موجودات زنده

Monk

راهب

Artificial

fertilization

باروری

مصنوعی

Sweet pea

نخود شیرین

Stochastic

beam search

جستجوی

پرتوی اتفاقی

Mechanism

سازوکار

Reversal

برگشت

Duplication

تکثیر

Genome

ژنوم

Mechanism

سازوکار

Baldwin

effect

اثر بالدوین

ف E
Update equation
معادله
به روزآوری
Hessian Matrix
ماتریس هسیان
Derivative
مشتق
Approximation
تقریب
Ridge
دماغه
Plateau
فلات
Random restart
شروع مجدد
تصادفی
Simulated annealing
شبه تاب کاری
Constrained optimization
بهینه سازی با محدودیت
Linear programming
برنامه ریزی خطی
Offline
برون خط
Online
برخط
Interleaving
یکی در میان

برای یافتن یک پیشینه یا کمینه f باید x را به نحوی پیدا کرد که گرادیان صفر شود (یعنی $\nabla f(x) = 0$). بنابراین، $g(x)$ در فرمول نیوتن همان $\nabla f(x)$ است و معادله به روزآوری می تواند به شکل ماتریسی برداری $x \leftarrow x - H^{-1}(x) \nabla f(x)$ نوشته شود که در آن $H_f(x)$ ماتریس هسیان، مشتق دوم است که عناصر H_{ij} آنها از $\delta^2 f / \delta x_i \delta x_j$ به دست می آید. از آنجا که هسیان H^2 ورودی دارد، روش نیوتن-رافسون در فضاهایی با ابعاد زیاد، پرهزینه است و به همین جهت، تقریبهای زیادی برای آن به وجود آمده است.

روشهای جستجوی محلی در فضاهای حالت پیوسته، به اندازه فضاهای گسسته از مشکلات پیشینه های محلی، دماغه ها و فلاتها رنج می برند. استفاده از شروع مجدد تصادفی و شبه تاب کاری اغلب می تواند مفید باشد. با این حال، فضاهای پیوسته با ابعاد زیاد مکانهای بزرگی هستند که در آنها معمولاً می توان به سادگی گم شد.

موضوع دیگری که باید به آن اشاره کنیم، بهینه سازی با محدودیت است. اگر جوابها باید محدودیتهای سختی را برای مقادیر متغیرها برآورده سازند در آن صورت یک مسأله بهینه سازی با محدودیت داریم. برای مثال، در مسأله تعیین محل فرودگاهها، می توانیم محلها را به داخل رومانی و روی خشکی محدود کنیم (نمی توان فرودگاه را وسط دریاچه قرار دارد). دشواری مسائل بهینه سازی با محدودیت به طبیعت محدودیتها و تابع هدف بستگی دارد. شناخته شده ترین دسته، دسته مسائل برنامه ریزی خطی است که در آن محدودیتها نامساویهای خطی هستند که یک ناحیه محدب شکل را تشکیل می دهند. مسائل برنامه ریزی خطی می تواند برحسب تعداد متغیرها در زمان چندجمله ای حل شود. علاوه بر این، مسائل با انواع مختلف محدودیتها و توابع هدف (برنامه ریزی درجه دوم، برنامه ریزی مخروطی مرتبه دوم و غیره) نیز مورد مطالعه قرار گرفته اند.

۴-۵ کارگزارهای جستجوی برخط و محیطهای ناشناخته

تا اینجا به کارگزارهایی پرداخته ایم که از الگوریتمهای جستجوی برون خط استفاده می کنند. این کارگزارها پیش از آن که قدم در دنیای واقعی بگذارند، یک راه حل کامل را محاسبه می کنند (شکل ۳-۱) و سپس، بدون مراجعه به ادراکاتشان این راه حل را اجرا می کنند. در مقابل، یک کارگزار جستجوی برخط^{۱۱} به صورت یکی در میان محاسبه و اقدام می کند: ابتدا یک اقدام انجام می دهد سپس محیط را مشاهده می کند و اقدام بعدی را محاسبه می کند. جستجوی برخط روش خوبی برای قلمروهای پویا و نیمه پویا می باشد (قلمروهایی که در آنها به ازای بی کاری و محاسبات بیش از حد طولانی، جریمه در نظر گرفته می شود). جستجوی برخط حتی روش خوبی برای قلمروهای اتفاقی است. به طور کلی، یک جستجوی برون خطی باید یک طرح اقتضایی بسیار عظیم برای تمامی اتفاقات ممکن ایجاد کرده باشد، در حالی که یک جستجوی برخط تنها باید آنچه در واقعیت اتفاق می افتد را در نظر بگیرد. برای مثال، به یک کارگزار بازی

^{۱۱} واژه "برخط" معمولاً در علم کامپیوتر برای ارجاع به الگوریتمهایی استفاده می شود که باید داده های ورودی را همان لحظه که دریافت می کنند پردازش کنند، به جای این که صبر کنند تا کل مجموعه داده های ورودی فراهم شود.

شطرنج توصیه درستی می‌شود که بسیار قبل‌تر از سنجش یک دور کامل بازی، اولین حرکتش را انجام دهد.

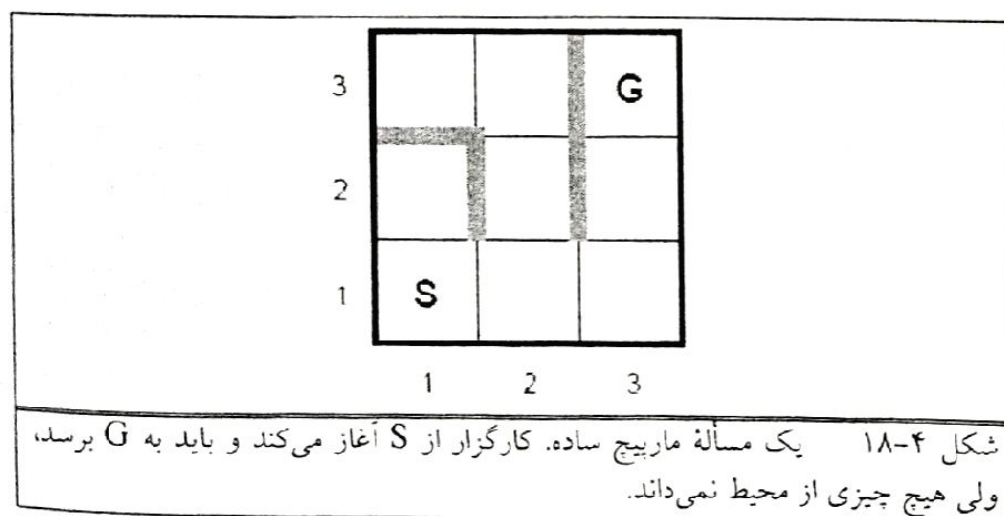
تفکر جستجوی برخط، در مسائل کاوشی که در آن حالتها و اقدامات برای کارگزار ناشناخته هستند، یک ضرورت می‌باشد. کارگزار در یک وضعیت نادانی برای تعیین اقدام بعدی‌اش باید از اقداماتش به عنوان تجربه استفاده کند و در نتیجه، باید محاسبه و اقدام را به صورت یکی در میان انجام دهد.

مثال استاندارد جستجوی برخط، روباتی است که در یک ساختمان جدید قرار داده می‌شود و باید آن را کاوش کند تا نقشه‌ای برای رفتن از A به B بیابد. روشهای فرار از مارپیچها (دانش مورد نیاز برای قهرمانان بلندپرواز عهد باستان) نیز نمونه‌هایی از الگوریتمهای جستجوی برخط هستند. با این حال کاوش، تنها محدود به کاوشهای مکانی نیست. یک نوزاد تازه متولد شده را در نظر بگیرید: وی دارای تعداد زیادی اقدام ممکن می‌باشد اما از نتیجه هیچ یک از آنها مطلع نیست و تنها تعداد کمی از حالتها را تجربه کرده است. کشف تدریجی چگونگی عملکرد نوزاد تا حدودی یک فرآیند جستجوی برخط می‌باشد.

مسائل جستجوی برخط

یک مسئله جستجوی برخط را نمی‌توان با یک فرآیند محاسباتی محض حل کرد، بلکه این کار را فقط یک کارگزار که اقدام انجام می‌دهد قادر به حل آن است. فرض می‌کنیم که کارگزار فقط موارد زیر را می‌داند:

- $ACTIONS(s)$: فهرستی از اقدامات مجاز در حالت s
- تابع گام-هزینه $c(s, a, s')$: توجه داشته باشید که این تابع تا زمانی که کارگزار نداند s' نتیجه کار می‌باشد نمی‌تواند استفاده شود.
- $GOAL-TEST(s)$

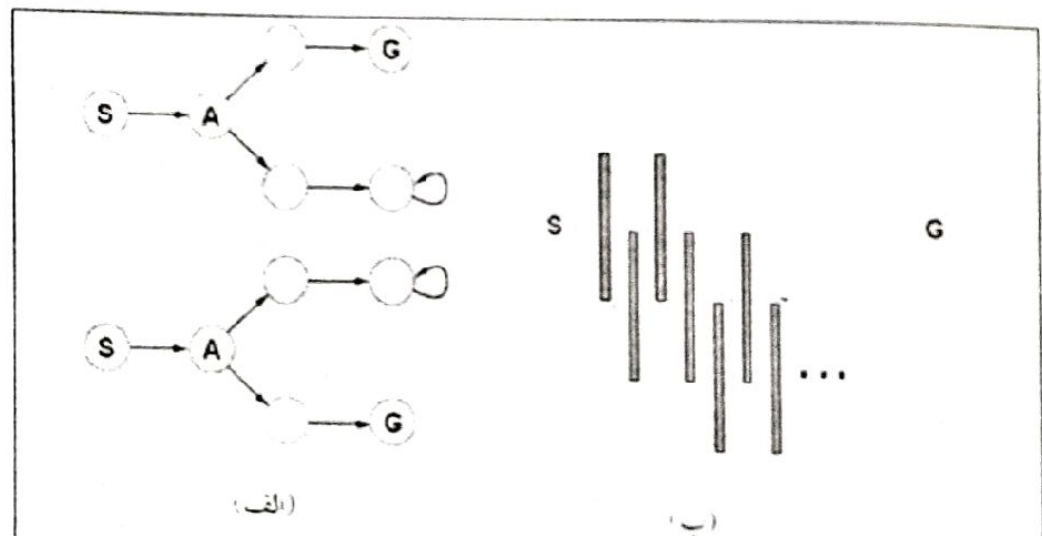


توجه داشته باشید که این کارگزار نمی‌تواند به پسینهای یک حالت دسترسی داشته باشد. مگر آن که تمامی اقدامها در آن حالت را به صورت واقعی امتحان کند. برای مثال، در مسئله مارپیچ که

$E \rightarrow F$	ف توصیه
Advise	توصیه
Exploration problem	مسئله کاوشی
State of ignorance	وضعیت نادانی
Experiment	آزمایش
Building	ساختمان
Agent	کارگزار
Map	نقشه
Escaping from labyrinth	فرار از مارپیچها
Aspiring heroes of antiquity	قهرمانان بلندپرواز عهد باستان
Spatial exploration	کاوشهای فضایی
Newborn baby	نوزاد تازه متولد شده
Pure computational process	محاسباتی محض
Maze	مارپیچ
Allowed	مجاز

ف $\Rightarrow$ E	
To try	امتحان کردن
Degree of ignorance	درجه نادانی
To reduce	کاهش دادن
Robot explorer	روبات کاوشی
Obstacle	مانع
To recognize	تشخیص دادن
Deterministic	معین
To relax	تعدیل کردن
Admissible heuristic	آروین قابل قبول
To estimate	تخمین زدن
Competitive ratio	نسبت رقابتی
Adversary argument	دعوی خصمانه
Accidentally	پیش‌بینی نشده
Irreversible	غیر قابل برگشت
Dead-end	بن‌بست
To prevent	اجتناب کردن

در شکل ۴-۱۸ نمایش داده شده است. کارگزار نمی‌داند که به سمت بالا رفتن از (I, I) به $(I, 2)$ ختم می‌شود و همچنین، پس از انجام این عمل نخواهد دانست که پایین رفتن وی را به (I, I) برمی‌گرداند. در بعضی از موارد این درجه نادانی قابل کاهش است. برای مثال، ممکن است یک روبات کاوشی از چگونگی اقدامات حرکتی‌اش آگاه باشد و تنها محل موانع را نداند. فرض می‌کنیم که کارگزار همیشه می‌تواند حالتی که قبلاً در آن بوده است را تشخیص دهد و فرض می‌کنیم که اقدامات معین هستند (این دو فرض در فصل ۱۷ تعدیل می‌شوند). بالاخره، کارگزار ممکن است به یک تابع آروین قابل قبول $h(s)$ دسترسی داشته باشد که فاصله را از حالت فعلی تا حالت هدف برآورد کند. برای مثال، در شکل ۴-۱۸ کارگزار ممکن است محل هدف را بداند و قادر باشد که از آروین فاصله منتهی استفاده کند. معمولاً هدف کارگزار، رسیدن به حالت هدف با کمترین هزینه است (هدف ممکن دیگر، کاوش کل محیط است). هزینه، کل هزینه مسیری است که کارگزار در واقعیت طی می‌کند. معمولاً این هزینه با هزینه طی مسیر کارگزاری با شناخت قبلی از فضای جستجو (یعنی کوتاه‌ترین مسیر واقعی یا کوتاه‌ترین کاوش کامل) می‌سنجند. به زبان الگوریتمهای برخط، نسبت این دو را نسبت رقابتی می‌نامند و ما ترجیح می‌دهیم تا حد امکان کوچک باشد.



شکل ۴-۱۹ (الف) دو فضای حالت که ممکن است یک کارگزار جستجوی برخط را به بن‌بست برسانند. هر کارگزاری ممکن است در حداقل یکی از فضاها شکست بخورد. (ب) یک محیط دوبعدی که می‌تواند باعث شود یک کارگزار جستجوی برخط یک مسیر اجباراً ناکارآمد را به سمت هدف دنبال کند. هر انتخابی که کارگزار انجام دهد، حریف مسیر وی را با یک دیوار بلند و باریک دیگر مسدود می‌کند، به نحوی که مسیری که طی می‌شود بسیار طولانی‌تر از بهترین مسیر ممکن می‌شود.

هر چند که ممکن است این خواسته یک تقاضای معقول به نظر برسد، ولی به سادگی می‌توان دید که در بعضی موارد بهترین نسبت رقابتی دست‌یافتنی، بی‌نهایت است. برای مثال، اگر بعضی اقدامات غیر قابل برگشت باشند، ممکن است جستجوی برخط به طور تصادفی به یک حالت

بن‌بست برسد که از آن هیچ حالت هدفی قابل دسترسی نیست.

شاید واژه "پیش‌بینی نشده" خیلی متقاعدکننده نباشد (بالاخره ممکن است الگوریتمی وجود داشته باشد که در هنگام کاوش به طور اتفاقی مسیر بن‌بست را انتخاب نکند). به طور دقیق‌تر ادعای ما این است که هیچ الگوریتمی نمی‌تواند در تمامی فضاهای حالت، از بن‌بست‌ها اجتناب کند. دو فضای حالت بن‌بست در شکل ۴-۱۹-الف را در نظر بگیرید. برای الگوریتم جستجوی برخطی که قبلاً از A و S گذشته است، هر دو فضای حالت یکسان به نظر می‌رسند. بنابراین، در هر دو فضا تصمیمات یکسانی باید اتخاذ کند. در نتیجه، در یکی از این دو شکست خواهد خورد. این نیز یک مثال از دعوای خصمانه است که در آن حریف، به هنگامی که کارگزار در حال کاوش است، فضای حالت را می‌سازد و می‌تواند هدفها و بن‌بست‌ها را هر جا که دوست دارد قرار دهد.

بن‌بست‌ها (پلکانها، سطوح شیب‌دار، صخره‌ها و تمامی انواع عوارض طبیعی زمین، ممکن است منجر به اقدامات غیر قابل برگشت شوند) یک مشکل واقعی برای روبات کاوشی محسوب می‌شوند. برای این که بتوانیم کار را پیش ببریم، فرض می‌کنیم که فضای حالت قابل کاوش امن است (یعنی از هر حالت دست‌یافتنی، حداقل یک حالت هدف قابل دسترسی باشد). فضاهای حالت با اقدامات قابل برگشت، مانند مارییچها و پازل‌های ۸ تایی را می‌توان همانند گرافهای بدون جهت در نظر گرفت و واضح است که قابل کاوش امن هستند.

در صورت وجود مسیرهای با هزینه نامحدود، حتی در محیطهای قابل کاوش امن هیچ نسبت رقابتی کران‌داری تضمین نمی‌شود. در محیطهایی که اقدامات غیر قابل برگشت دارند این مسأله به راحتی قابل اثبات است و در مورد بازگشت‌پذیر نیز همان‌طور که در شکل ۴-۱۹-ب مشاهده می‌شود، این مسأله صحیح است. به همین دلیل معمولاً کارایی الگوریتمهای جستجوی برخط علاوه بر حسب عمق کم عمق‌ترین هدف، بر حسب اندازه کل فضای حالت نیز بیان می‌شود.

کارگزارهای جستجوی برخط

پس از هر اقدام، کارگزار برخط، یافته‌ای را دریافت می‌کند که به وی می‌گوید به چه حالتی رسیده است. با استفاده از این اطلاعات، کارگزار می‌تواند نقشه‌اش را از محیط تکمیل کند. از نقشه فعلی برای این تصمیم که در گام بعدی باید به کجا رفت، استفاده می‌شود. یکی درمیان بودن برنامه‌ریزی و اقدام بدین معنی است که الگوریتمهای جستجوی برخط کاملاً با الگوریتمهای جستجوی برون‌خط که قبلاً دیده‌ایم متفاوت هستند. برای مثال، الگوریتمهای برون‌خط از قبیل A^* این قابلیت را دارند که با گسترش یک گره را در یک بخش فضا بلافاصله گرهی را در بخش دیگری از فضا گسترش می‌دهند، زیرا گسترش گره شامل اقدامات شبیه‌سازی شده است، نه اقدامات واقعی. ولی یک الگوریتم برخط می‌تواند فقط گرهی را که به طور فیزیکی اشغال کرده است گسترش دهد. برای اجتناب از جابه‌جایی در عرض درخت به منظور گسترش گره بعدی، به نظر می‌رسد بهتر باشد که گره‌ها را به ترتیب محلی گسترش دهیم. جستجوی اول عمق دقیقاً همین ویژگی را دارد، زیرا گره بعدی که گسترش می‌یابد، فرزند گره قبلی است که گسترش یافته است (به استثنای زمانی که حرکت روبه عقب صورت می‌گیرد).

یک کارگزار جستجوی اول عمق برخط در شکل ۴-۲۰ نمایش داده شده است. این کارگزار، نقشه‌اش را در یک جدول به نام $result[a, s]$ نگهداری می‌کند. این جدول حالت

$E \rightarrow$	کارایی
Performance	عمق
Depth of the shallowest goal	کم عمق ترین
	هدف
Percept	ادراک
Map	نقشه
Environment	محیط
Interleaving	یکی در میان
	بودن
Offline algorithm	الگوریتم
	برون خط
Simulated actions	اقدامات
	شبیه سازی شده
In a local order	به ترتیب محلی
Property	ویژگی
Backtracking	حرکت روبه عقب
Arbitrarily inefficient	اجباراً ناکارآمد
Space	فضا
Agent	کارگزار
To block	مسدود کردن

$E \Rightarrow$ ف	کارگزار
Online depth-first agent	جستجوی اول عمق
کارگزار	جستجوی اول عمق
جستجوی اول عمق	جستجوی اول عمق
Action	اقدام
Exploration	کاوش
To try	امتحان کردن
Table	جدول
جدول	جدول
Predecessor states	حالت‌های پیشین
Backtrack	عقب‌گرد
عقب‌گرد	عقب‌گرد
Maze	مارپیچ
مارپیچ	مارپیچ
Bounded	کران‌دار
کران‌دار	کران‌دار
Link	پیوند
پیوند	پیوند
State space	فضای حالت
فضای حالت	فضای حالت
Optimal	بهینه
بهینه	بهینه
Agent's competitive ratio	نسبت رقابتی
نسبت رقابتی	نسبت رقابتی
Variant	گونه، نوع
گونه، نوع	گونه، نوع
Uniform tree	درخت یکنواخت
درخت یکنواخت	درخت یکنواخت
Constant	ثابت
ثابت	ثابت

حاصل از اجرای اقدام a در حالت s را ثبت می‌کند. کارگزار اقدام‌های کاوش نشده از حالت فعلی را امتحان می‌کند. مشکل زمانی پیش می‌آید که کارگزار تمامی اقدامات یک حالت را امتحان کرده باشد. در جستجوی اول عمق برون‌خط تنها کافی است این حالت را از صف خارج کنیم. در یک جستجوی برخخط، کارگزار باید به صورت فیزیکی به عقب برگردد. در جستجوی اول عمق این کار به معنی برگشت به آخرین حالتی است که کارگزار از آن به حالت فعلی وارد شده است. برای انجام آن باید برای هر حالت، فهرست حالت‌های پیشینی که کارگزار هنوز به آنها عقب‌گرد نکرده است در جدولی نگهداری شود. اگر دیگر حالتی برای کارگزار باقی نمانده باشد که به آن عقب‌گرد کند، جستجویش کامل شده است.

```

function ONLINE-DFS-AGENT( $s'$ ) returns an action
inputs:  $s'$ , a percept that identifies the current state
static: result, a table, indexed by action and state, initially empty
        unexplored, a table that lists, for each visited state, the actions not yet tried
        unbacktracked, a table that lists, for each visited state, the backtracks not yet tried
         $s$ ,  $a$ , the previous state and action, initially null

if GOAL-TEST( $s'$ ) then return stop
if  $s'$  is a new state then unexplored[ $s'$ ]  $\leftarrow$  ACTIONS( $s'$ )
if  $s$  is not null then do
    result[ $a$ ,  $s$ ]  $\leftarrow s'$ 
    add  $s$  to the front of unbacktracked[ $s'$ ]
if unexplored[ $s'$ ] is empty then
    if unbacktracked[ $s'$ ] is empty then return stop
    else  $a \leftarrow$  an action  $b$  such that result[ $b$ ,  $s'$ ] = POP(unbacktracked[ $s'$ ])
else  $a \leftarrow$  POP(unexplored[ $s'$ ])
 $s \leftarrow s'$ 
return  $a$ 

```

شکل ۴-۲۰ یک کارگزار جستجوی برخخط که از کاوش اول عمق استفاده می‌کند. این کارگزار تنها در فضاهای جستجوی دوطرفه قابل استفاده است.

پیشنهاد می‌کنیم که با اعمال $ONLINE-DFS-AGENT$ به مارپیچ شکل ۴-۱۸ آن را دنبال نمایید. به سادگی می‌توان دید که کارگزار در بدترین حالت، حرکت عرضی بین پیوندهای موجود در فضاهای حالت را دقیقاً با دوبار عبور به پایان می‌رساند. این روش در مورد مسائل کاوش، بهینه می‌باشد. در حالی که در مسائل پیدا کردن هدف، هنگامی که یک هدف در نزدیکی حالت شروع باشد، در صورت انتخاب مسیر اشتباه، نسبت رقابتی کارگزار ممکن است بد شود. یک نوع جستجوی عمیق‌شونده تکراری برخخط این مسأله را حل می‌کند. نسبت رقابتی این کارگزار در یک محیط به شکل یک درخت یکنواخت، ثابت کوچکی است.

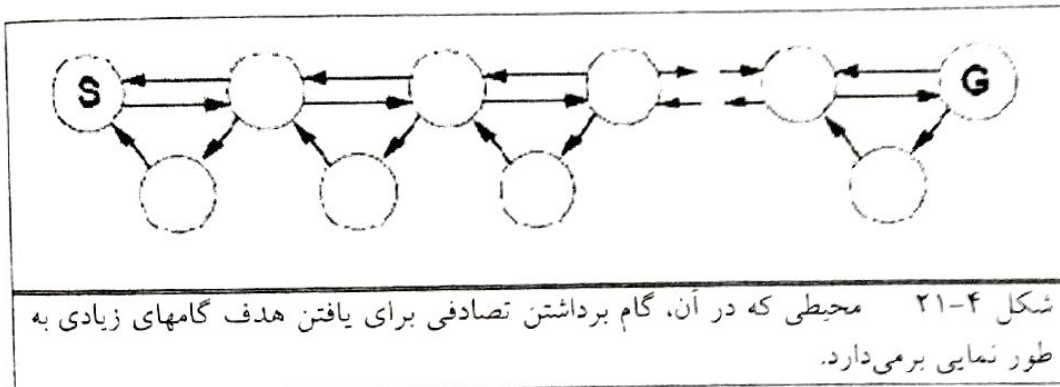
$ONLINE-DFS-AGENT$ به علت روش بازگشت به عقبش، فقط در فضاهای حالتی عمل می‌کند که اقدامات، برگشت‌پذیر هستند. الگوریتم‌های اندکی پیچیده‌تر وجود دارند که در فضاهای حالت کلی عمل می‌کنند، ولی هیچ کدام از آنها نسبت رقابتی کران‌داری ندارد.

جستجوی محلی برخخط

همانند جستجوی اول عمق، جستجوی تهنوردی نیز در گسترش گره‌هایش دارای ویژگی

محلی بودن می‌باشد. در حقیقت، از آنجا که تنها یک حالت فعلی در حافظه نگهداری می‌شود، جستجوی تپه‌نوردی یک الگوریتم جستجوی برخط است! متأسفانه این روش، در شکل ابتدایی آن خیلی سودمند نمی‌باشد زیرا کارگزار را در بیشینه‌های محلی رها می‌کند. علاوه بر این، از شروع مجدد تصادفی نیز نمی‌توان استفاده کرد، زیرا کارگزار نمی‌تواند خودش را به یک حالت جدید منتقل کند.

در کاوش محیط به جای شروع مجدد تصادفی، می‌توان از گام برداشتن تصادفی استفاده کرد. گام برداشتن تصادفی یکی از اقدامات موجود حالت فعلی را به صورت تصادفی انتخاب می‌کند. می‌توان به اقداماتی که تاکنون امتحان نشده‌اند اولویت بالاتری داد. به سادگی می‌توان ثابت کرد که اگر فضا متناهی باشد، گام‌برداری تصادفی در نهایت یک هدف را پیدا می‌کند یا کاوشهایش را کامل می‌نماید.^{۱۰} از طرف دیگر، این فرآیند ممکن است خیلی کند باشد. شکل ۴-۲۱ محیطی را نشان می‌دهد که در آن، گام برداشتن تصادفی برای یافتن هدف گامهای زیادی به طور نمایی برمی‌دارد، زیرا در هر گام احتمال پیش‌روی به عقب دو برابر پیش‌روی به جلو است. اگرچه این مثال ساختگی است ولی فضاهای حالت بسیاری در دنیای واقعی وجود دارد که توبولوژی آنها باعث ایجاد چنین تله‌هایی برای گام برداشتن تصادفی می‌شود.



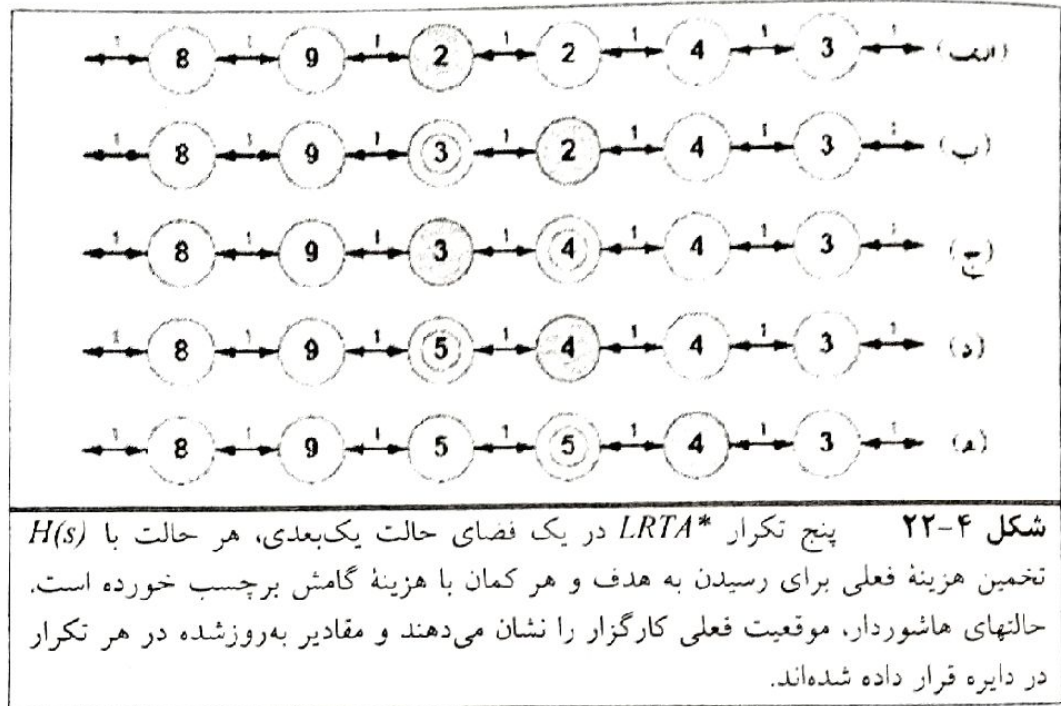
اگر برای تقویت روش تپه‌نوردی، به جای تصادفی بودن، از حافظه استفاده کنیم، رویکرد مؤثرتری خواهیم داشت. فکر اصلی در این روش، نگهداری $H(s)$ یا "بهترین برآورد فعلی" هزینه رسیدن به هدف از هر حالتی است که از آن گذشته‌ایم. $H(s)$ با برآورد آروین $h(s)$ آغاز می‌شود و به تدریج که کارگزار در فضای حالت تجربه کسب می‌کند به‌روزرآوری می‌شود. شکل ۴-۲۲ یک شکل ساده را در یک فضای حالت یک‌بعدی نمایش می‌دهد. در (الف) به نظر می‌رسد که کارگزار در یک کمینه محلی مسطح در حالت هاشوردار گیر کرده است. به جای این که کارگزار سرجایش باقی بماند، باید براساس برآورد هزینه فعلی همسایگانش، بهترین مسیر به سمت هدف را دنبال کند. هزینه تخمینی رسیدن به هدف از طریق همسایه‌ای مانند s' هزینه رسیدن به s' به‌علاوه هزینه تخمینی رسیدن به یک هدف از آنجاست (یعنی $c(s, a, s') + H(s')$). در این مثال، دو اقدام وجود دارد که دارای هزینه‌های تخمینی $۱+۹$ و $۱+۲$ هستند. بنابراین، به نظر می‌رسد که بهتر است

ف	E
Hill climbing	تپه‌نوردی
Online local search	جستجوی محلی برخط
Depth-first search	جستجوی اول عمیق
Property	ویژگی
Local maximum	بیشینه محلی
Random restart	شروع مجدد تصادفی
Environment	محیط
Priority	اولویت
Infinite	متناهی
Complete	کامل
Random walk	گام‌برداری تصادفی
Backward progress	پیشرفت رو به عقب
Contrived	ساختگی
Trap	تله

^{۱۰} در فضاهای نامتناهی، کار بسیار مشکل‌تر است. روش گام‌برداشتن تصادفی در شبکه‌های یک‌بعدی و دوبعدی، کامل می‌باشد ولی در شبکه‌های سه‌بعدی کامل نیستند! در حالت سه‌بعدی احتمال این که در حال گام برداشتن مجدداً به حالت شروع برگردیم فقط حدود ۰.۳۴-۰.۵ می‌باشد. (به منظور آشنایی کلی با این مبحث به (Hughes, 1995) مراجعه کنید).

Estimated costs	هزینه‌های تخمینی
Shaded	هاشوردار
Optimistic	خوش‌بینانه
Step	گام
Distance	فاصله
Update	به‌روزرسانی
Local minimum	کمینه محلی
To escape	فرار کردن
Goal	هدف
Arc	کمان
Label	برچسب
Design	طرح
Learning real-time A*	A* یادگیرنده
Optimism under uncertainty	خوش‌بینی تحت عدم قطعیت
Infinite	نامتناهی
Astray	سرگردان
Guaranteed	تضمین‌شده

به سمت راست حرکت کنیم. اکنون مشخص شده است که برآورد هزینه ۲ برای حالت هاشوردار بیش از حد خوش‌بینانه بوده است. از آنجا که هزینه بهترین حرکت ۱ بوده است و به حالتی منجر شده است که حداقل ۲ گام از هدف فاصله دارد، حالت هاشوردار باید حداقل ۳ گام از هدف فاصله داشته باشد. بنابراین، همان‌طور که در شکل ۴-۲۲ ب نشان داده شده است، H آن باید بر همین اساس به‌روزرسانی شود. با ادامه این فرآیند، کارگزار دو برابر بیشتر به عقب و جلو حرکت می‌کند و هر بار H را به‌روزرسانی و کمینه محلی را "سطح" می‌کند تا زمانی که در نهایت از سمت راست آن فرار کند.



کارگزاری که این طرح را اجرا می‌کند، A^* بی‌درنگ یادگیرنده ($LRTA^*$) نامیده می‌شود و در شکل ۴-۲۳ نشان داده شده است. همانند $ONLINE-DFS-AGENT$ این کارگزار نیز با استفاده از جدول $result$ نقشه‌ای از محیط تهیه می‌کند. کارگزار هزینه تخمینی حالتی که هم اکنون از آن خارج شده است را به‌روزرسانی می‌کند. سپس، براساس تخمین‌های هزینه فعلی‌اش، حرکتی را که به ظاهر بهترین است انتخاب می‌کند. یک نکته مهم این است همیشه فرض بر این است که اقداماتی که تا این لحظه در حالت s امتحان نشده‌اند، با کمترین هزینه ممکن $h(s)$ به هدف منتهی می‌شوند. این خوش‌بینی تحت عدم قطعیت، کارگزار را تشویق به کاوش مسیرهای جدید و احتمالاً امیدوار می‌کند.

برای یک کارگزار $LRTA^*$ ، پیدا کردن هدف در هر محیط متناهی قابل کاوش امن، تضمین شده است. با این حال، بر خلاف A^* برای فضاهای حالت نامتناهی کامل نیست (حالت‌هایی وجود دارد که ممکن است برای همیشه در آنها سرگردان شود). این روش در بدترین حالت می‌تواند یک محیط با n حالت را در $O(n^2)$ گام کاوش کند، ولی اغلب بهتر از این عمل می‌کند. کارگزار $LRTA^*$ تنها یکی از اعضای خانواده بزرگی از کارگزارهای برخلاف است که می‌توان آنها را با تعیین قانون انتخاب اقدام و قانون به‌روزرسانی به طرق مختلف بیان کرد. این

خانواده که در اصل برای محیطهای اتفاقی ایجاد شده‌اند را در فصل ۲۱ مورد بحث قرار خواهیم داد.

یادگیری در جستجوی برخط

نادانی اولیه کارگزارهای جستجوی برخط، موقعیتهای متعددی برای یادگیری فراهم می‌کنند. اول از همه، کارگزار با ثبت تمامی تجربیاتش، نقشه محیط را به‌سادگی یاد می‌گیرد، یا به عبارت دقیقتر، نتیجه هر اقدام در هر حالت را یاد می‌گیرد. (توجه داشته باشید که فرض محیطهای معین بدین معنی است که تنها یک تجربه برای هر اقدام کافی است.) دوم، کارگزارهای جستجوی محلی برآوردهای دقیقتری را از ارزش هر حالت، با استفاده از قوانین به‌روزرسانی محلی همانند آنچه در $LRTA^*$ اتفاق می‌افتد، به‌دست می‌آورند. در فصل ۲۱، خواهیم دید که به شرطی که کارگزار فضای حالت را به‌طور صحیحی مورد کاوش قرار دهد، این به‌روزرسانیها در نهایت به مقادیر دقیق حالتها منجر می‌شوند. به محض تعیین مقادیر دقیق، به سادگی با حرکت به پسین بالاترین مقدار، تصمیمات بهینه عملی می‌شوند. (یعنی در این حالت، تپه‌نوردی محض یک راهبرد بهینه است.)

$E \rightarrow$	ف
Member	عضو
Family	خانواده
Stochastic environment	محیط اتفاقی
Ignorance	نادانی
Opportunity	موقعیت
Learning	یادگیری
Outcome of each action	نتیجه هر اقدام
Experience	تجربه
Exact values	مقادیر دقیق
Optimal decisions	تصمیمات بهینه
Highest-valued	بالاترین مقدار
Successor	پسین
Suggestion	پیشنهاد
Pure hill climbing	تپه‌نوردی خالص
Bright	باهوش
To follow	پی‌گیری کردن

```

function LRTA*-AGENT( $s'$ ) returns an action
inputs:  $s'$ , a percept that identifies the current state
static: result, a table, indexed by action and state, initially empty
         $H$ , a table of cost estimates indexed by state, initially empty
         $s$ , a, the previous state and action, initially null

if GOAL-TEST( $s'$ ) then return stop
if  $s'$  is a new state (not in  $H$ ) then  $H[s'] \leftarrow h(s')$ 
unless  $s$  is null
    result[a,  $s$ ]  $\leftarrow s'$ 
     $H[s] \leftarrow \min_{b \in \text{ACTIONS}(s)} \text{LRTA}^*\text{-COST}(s, b, \text{result}[b, s], H)$ 
     $a \leftarrow$  an action  $b$  in  $\text{ACTIONS}(s')$  that minimizes  $\text{LRTA}^*\text{-COST}(s', b, \text{result}[b, s'], H)$ 
     $s \leftarrow s'$ 
return a

function LRTA*-COST( $s, a, s', H$ ) returns a cost estimate
if  $s'$  is undefined then return  $h(s)$ 
else return  $c(s, a, s') + H[s']$ 
    
```

شکل ۴-۲۲ $LRTA^*\text{-Agent}$ بر اساس ارزشهای حالتهای همسایه‌اش که در ضمن حرکت کارگزار در فضای حالت به‌روزرسانی می‌شوند، یک اقدام را انتخاب می‌کند.

اگر پیشنهاد ما را برای پی‌گیری رفتار $ONLINE\text{-DFS-AGENT}$ در محیط شکل ۴-۱۸ دنبال کرده باشید، متوجه شده‌اید که این کارگزار چندان هم باهوش نیست. برای مثال، پس از این که متوجه شده است که اقدام UP از $(1, 1)$ به $(1, 2)$ می‌رود، هنوز هیچ تصویری در این زمینه ندارد که اقدام $Down$ به $(1, 1)$ برمی‌گردد یا این که اقدام UP همچنین از $(2, 1)$ به $(2, 2)$ و از $(2, 2)$ به $(2, 3)$ می‌رود و غیره. به‌طور کلی ترجیح می‌دهیم کارگزار یاد بگیرد که UP ، مختصات y را افزایش می‌دهد مگر این که دیواری در سر راه قرار داشته باشد و $Down$ ، y را کاهش می‌دهد و غیره. برای این که این موضوع اتفاق بیفتد، به دو چیز نیاز داریم. اول، به یک بازنمایی رسمی و صراحتاً قابل اعمال برای این نوع قوانین کلی نیاز داریم. تا اینجا این اطلاعات را درون جعبه

$E \rightarrow$	ف
Coordinates	مختصات
Black box	جعبه سیاه
Successor function	تابع پسین
Optimality	بهینگی
Expensive	هزینه‌بر
Best-first search	جستجوی اول‌بهترین
Unexpanded node	گره گسترش‌نیافته
Estimate	برآورد
Greedy	حرصی
To guarantee	تضمین کردن
Consistent	سازگار
Subproblem	زیرمسئله
Hill climbing	تپهنوردی
Complete-state formulation	تدوین حالت کامل
Stochastic	اتفاقی
Cooling	سردسازی
Crossover	پیوند
Mutation	جهش

سیاهی به نام تابع پسین پنهان کرده‌ایم. بخش سوم به این مسأله اختصاص یافته است. دوم، به الگوریتمهایی نیاز دارید که بتوانند قوانین عمومی مناسبی را از روی مشاهدات کارگزار، بسازند. به این الگوریتمها در فصل ۱۸ پرداخته می‌شود.

۶-۴ خلاصه

این فصل کاربرد آروینها در کاهش هزینه‌های جستجو را بررسی کرد. به الگوریتمهایی که از آروین استفاده می‌کنند پرداختیم و دریافتیم که حتی با آروینهای خوب بهینگی، از نظر هزینه جستجو، هزینه‌بر است.

• جستجوی اول‌بهترین، همان $GRAPH_SEARCH$ است با این تفاوت که گرههای گسترش‌نیافته‌ای که (براساس مقیاس خاصی) کمترین هزینه را دارند برای گسترش انتخاب می‌شوند. الگوریتمهای اول‌بهترین معمولاً از تابع آروین $h(n)$ استفاده می‌کنند که هزینه یک راه‌حل از n را برآورد می‌کند.

• جستجوی اول‌بهترین حریصانه گرههایی که کمترین $h(n)$ دارند را گسترش می‌دهد. این روش بهینه نیست ولی اغلب مؤثر است.

• جستجوی A^* گرههایی با کمترین $f(n) = g(n) + h(n)$ را گسترش می‌دهد. A^* کامل و بهینه است به شرطی که تضمین کنیم که $h(n)$ قابل قبول (برای $TREE_SEARCH$) یا سازگار (برای $GRAPH_SEARCH$) است. پیچیدگی فضایی A^* هنوز یک مانع محسوب می‌شود.

• کارایی الگوریتمهای جستجوی آروین به کیفیت تابع آروین بستگی دارد. آروینهای خوبی را گاهی می‌توان با ساده‌سازی تعریف مسأله، یا با پیش‌محاسبه هزینه‌های جواب مسائل فرعی در پایگاه اطلاعاتی الگو یا با یادگیری تجربی از مسأله ساخت.

• $RBFS$ و SMA^* الگوریتمهای قدرتمند و بهینه‌ای هستند که از حافظه محدودی استفاده می‌کنند. اگر به آنها زمان کافی داده شود، می‌توانند مسائلی را حل کنند که A^* به خاطر کمبود حافظه نمی‌توانست حل کند.

• روشهای جستجوی محلی مانند تپهنوردی در تدوین حالت کامل عمل می‌کنند و تنها تعداد اندکی گره را در حافظه نگهداری می‌کنند. الگوریتمهای اتفاقی زیادی به وجود آمده‌اند، مانند شبه تاب‌کاری که اگر برنامه زمانی سردسازی آنها مناسب باشد، جوابهای بهینه برمی‌گردانند. بسیاری از روشهای جستجوی محلی برای حل مسائل در فضاهای پیوسته نیز می‌توانند استفاده شوند.

• الگوریتم ژنتیک جستجوی تپهنوردی اتفاقی است که در آن جمعیت زیادی از حالتها نگهداری می‌شود. حالتها جدید توسط جهش و پیوند که جفت‌هایی از حالتها جمعیت را با هم ترکیب می‌کنند، به وجود می‌آیند.

• مسائل کاوشی زمانی مطرح می‌شوند که کارگزار هیچ آگاهی از حالتها و اقدامات محیطش نداشته باشد. در مورد محیطهای قابل کاوش امن، کارگزارهای جستجوی برخط می‌توانند با تهیه نقشه، اگر هدفی وجود داشته باشد آن را بیابند. به‌روزرسانی برآوردهای آروین براساس

تجربه، روش مؤثری برای فرار از کمینه‌های محلی فراهم می‌کند.

ف $\Leftrightarrow$ E

Exploration
problems